

ion_gym

An independent, self-contained toolkit for ion-optics simulation

User Manual — first edition

Clowers Research Group
Washington State University

Generated 2026-09-04

Contents

Contents	2
1. What this toolkit is	4
1.1 What it is not.....	4
1.2 The two ways to drive it.....	4
2. Quick start	4
1.3 Installation	5
2.1 Viewing the trajectories.....	5
2.2 The same flight in the application — a worked example.....	5
3. The specification file	10
4. Geometry	11
4.1 Declared primitives	11
4.2 Importing STL meshes	11
4.3 File organization.....	12
4.4 Symmetry and mirror planes	12
5. Solving the field	12
5.1 Per-electrode bases and superposition.....	13
5.2 The iterative solvers	13
6. From potential to field	13
6.1 The two methods.....	13
6.2 When each is appropriate	14
7. Flying ions.....	14
7.1 The integrator	14
7.2 Time-varying drives	15
7.3 Collisions with background gas	15
7.4 Units	15
8. The pseudopotential view.....	16
9. Reading results.....	16
10. Limitations.....	17
11. Dashboard reference (tab by tab)	18
Ion Source.....	18
Voltages	22
Physics.....	26
Display	27
Impact Analysis	29
Geometry Import	29
Status	30
Multi FA	32

Config	32
View (plot pane)	34
Analysis (plot pane)	34
Thermal (plot pane)	35
PE Surface (plot pane)	35
Field Slice (plot pane)	35
Raster (plot pane)	36
Cache (plot pane)	36
Worked example captures	36
11.1 The geometry editor (/editor)	42
11.2 The 3-D Flight Viewer (/flight)	45
12. References	47
Appendix A. The instrument specification, shape by shape	48
A.1 The skeleton	48
A.2 The shape primitives	49
A.3 Signed frames: origin_mm	50
A.4 A symmetry example	50
A.5 A fully defined shape set: the analytical quadrupole	51

1. What this toolkit is

ion_gym computes electrostatic, electrodynamic, and radio-frequency fields for ion optics simulations. It is a Python toolkit: geometry, voltages, drive waveforms, gas conditions and the ion source are all declared in a JSON specification file, solved with its own field solver, and integrated with its own trajectory kernel. It does not require, wrap, or call any external ion-optics package.

The design goal is simplicity and reproducibility. A specification file plus its geometry resources is a complete, portable description of a simulation: the same file gives the same field and the same trajectories on another machine. Every quantity the solver uses is declared somewhere you can read, rather than being set in a graphical interface and lost.

1.1 What it is not

Being explicit about the boundaries is more useful than a feature list:

- It is not a general electromagnetic solver. It solves Laplace's equation for electrostatics on a uniform Cartesian or cylindrical grid. **There are no magnetic fields, no space charge, and no dielectrics.**
- It is not a replacement for established commercial ion-optics packages. Where the two disagree, that disagreement is a finding to be investigated, not a result.
- It is not a CAD package. Though a shaped editor exists as an ancillary control, geometry is generally built from declared primitives (e.g., circles, rectangles, and polygons_ or imported as STL meshes produced elsewhere.
- It does not model gas dynamics. Collisions use a statistical model at a declared pressure and temperature though a uniform flow vector can be applied (API only at this time).

1.2 The two ways to drive it

Everything below can be done either from the browser-based application or from a script. The application is a viewer and an editor for the same specification files the scripts consume; it holds no configuration of its own. **Anything you can set in the interface is a field in the file.**

2. Quick start

A minimal session loads a specification, builds the field, and flies ions:

```
from ion_gym.io.sim_spec import SimSpec
from ion_gym.physics.sim_build import build_run

spec = SimSpec.from_json('examples/einzel_round_r-z.json')
model, fly, col_names, births = build_run(spec, verbose=True)

traj, summary = fly(0)          # fly ion 0
print(col_names)               # what the columns of traj mean
print(summary['tof'], summary['ke_end'])
```

Load specifications by **path**, not by pasting their text. A specification loaded from a path records where it came from, so relative resources — an STL directory, a births file — resolve against the specification rather than against your current working directory. This is what makes a specification plus its resources portable.

The returned `fly` is a function: calling `fly(i)` flies ion `i` and returns its trajectory array and a summary dictionary. Building the model and flying ions are separate steps because the field is expensive and the flight is cheap; one solved field serves any number of ions.

1.3 Installation

The toolkit installs as a normal Python package from the repository root. The core package needs only the scientific stack; the browser application and the notebooks pull extra dependencies, grouped as extras:

```
python -m pip install -e .           # core only
python -m pip install -e ".[editor]" # + the browser application
python -m pip install -e ".[notebooks]" # + the teaching notebooks
python -m pip install -e ".[all]"     # everything
```

The first run after installation is slower than every later one: the physics kernels compile on first use and are cached to disk, and field solves are banked so a repeated build is a cache hit. Nothing needs to be configured; paths are repo-relative and the package never writes outside its own tree.

2.1 Viewing the trajectories

A raw trajectory array is rarely what you want to look at as a table. However, to view the flight, hand the flown paths back to the framework's own renderer — the same path every figure in this manual took:

```
from ion_gym.viz.viz_core import scene_from_simspec, report

trajs, fates = [], []
for i in range(len(births)):
    tr, summ = fly(i)
    if tr is not None and len(tr):
        trajs.append(tr)
        fates.append(str(summ.get('kind', '')))

sc = scene_from_simspec(spec, model, field='phi',
                       trajs=trajs, fates=fates)
report([sc], 'outputs/einzel_quickstart', name='einzel quickstart')
```

The report draws the electrodes as solved, the equipotentials of the field the ions actually flew through, and every path colored by how its flight ended — and for a three-dimensional model it does so in each principal plane, because a single projection of a 3-D instrument hides exactly the off-axis behavior you are usually looking for.

2.2 The same flight in the application — a worked example

The application runs the identical loop with buttons instead of code. This walk uses the same shipped deck the script above loads — “Einzel Lens (r-z)” — and every screenshot below is a real capture of that session, one figure per step.

Step 1 — load the deck. Open the Config tab, pick “Einzel Lens (r-z)” in the example loader — picking stages the example and shows its solve cost before you commit — then press Load example.

[Fly](#)
[Stop](#)
[Clear ions](#)
[Recompute field](#)
[Reset app](#)

[Ion Source](#)
[Voltages](#)
[Physics](#)
[Display](#)
[Impact Analysis](#)
[Geometry Import](#)
[Status](#)
[Multi FA](#)
[Config](#)

ions per m/z
 seeded run ☐ seed
 Total flown = this x number of listed masses

m/z list (Da, comma-separated)
 Comma-delimited: '120, 240' flies BOTH masses

Each m/z is flown with its OWN block of ions per m/z ions — e.g. 50 with 3 masses flies 150, 50 of each. These govern Basic and Advanced.

[Basic](#)
[Advanced](#)
[Stations](#)
[Stats](#)

birth distribution

start x (mm)
 start y (mm)
 start z (mm)

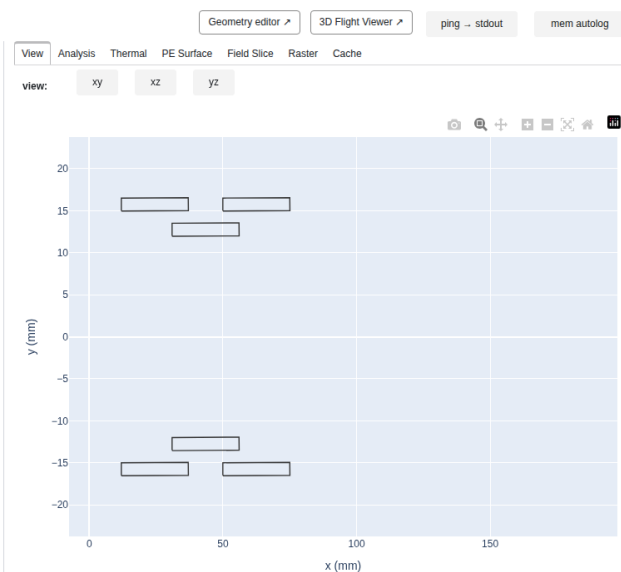
birth cloud size (mm)

KE min (eV)
 KE max (eV)
 temperature (K)

launch direction (KE is along this vector):

launch dir x
 launch dir y
 launch dir z

[Clear ions](#)



Step 2 -- Ion Source tab after loading Einzel Lens (r-z): the deck declares 12 ions on a 3 mm disc at 100 eV along +x, m/z 120. The m/z list is COMMA-DELIMITED: '120' flies one mass; '120, 240' flies a full block of ions at each listed mass.

Step 3 — check the flight conditions. The Physics tab carries the integration conditions the deck ships: a 2 ns time step, a 60 μ s maximum flight time, and collisions off (vacuum). What this tab shows is what the integrator receives.

[Fly](#)
[Stop](#)
[Clear ions](#)
[Recompute field](#)
[Reset app](#)

[Ion Source](#)
[Voltages](#)
[Physics](#)
[Display](#)
[Impact Analysis](#)
[Geometry Import](#)
[Status](#)
[Multi FA](#)
[Config](#)

[Gas](#)
[Integration](#)
[Bounds](#)

time step Δt (ns)
 Integration step the deck declares

max flight time (μ s)
 Ions still airborne here stop as 'time out'

record every N steps

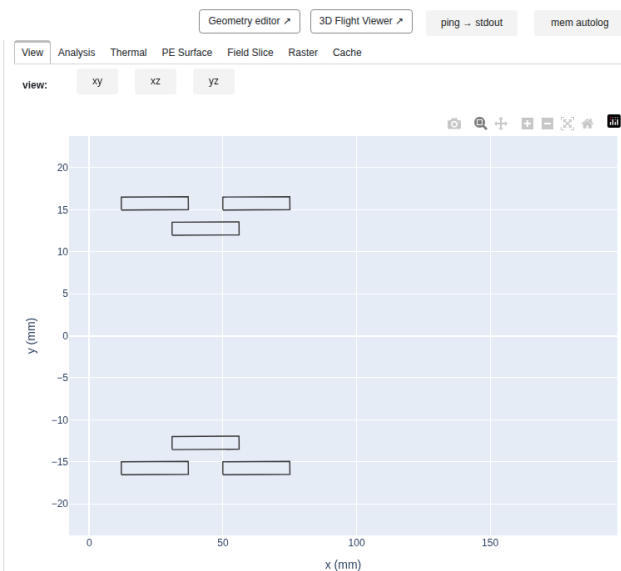
max recorded points per ion

extra recorded quantities

long-run storage — trajectories can be large; turn storage off to just fly ions and keep only their fates. (Clear stored runs is on Config > Runs > maintenance.)

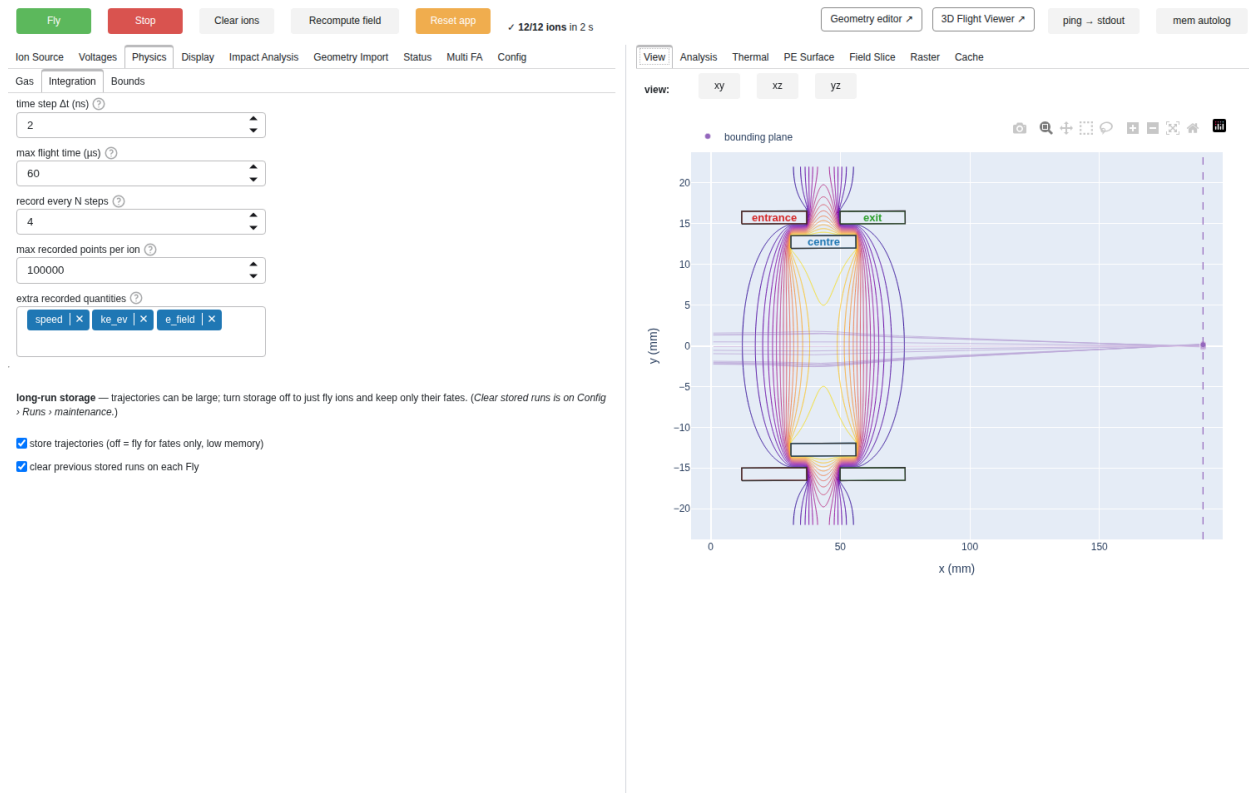
☒ store trajectories (off = fly for fates only, low memory)

☒ clear previous stored runs on each Fly



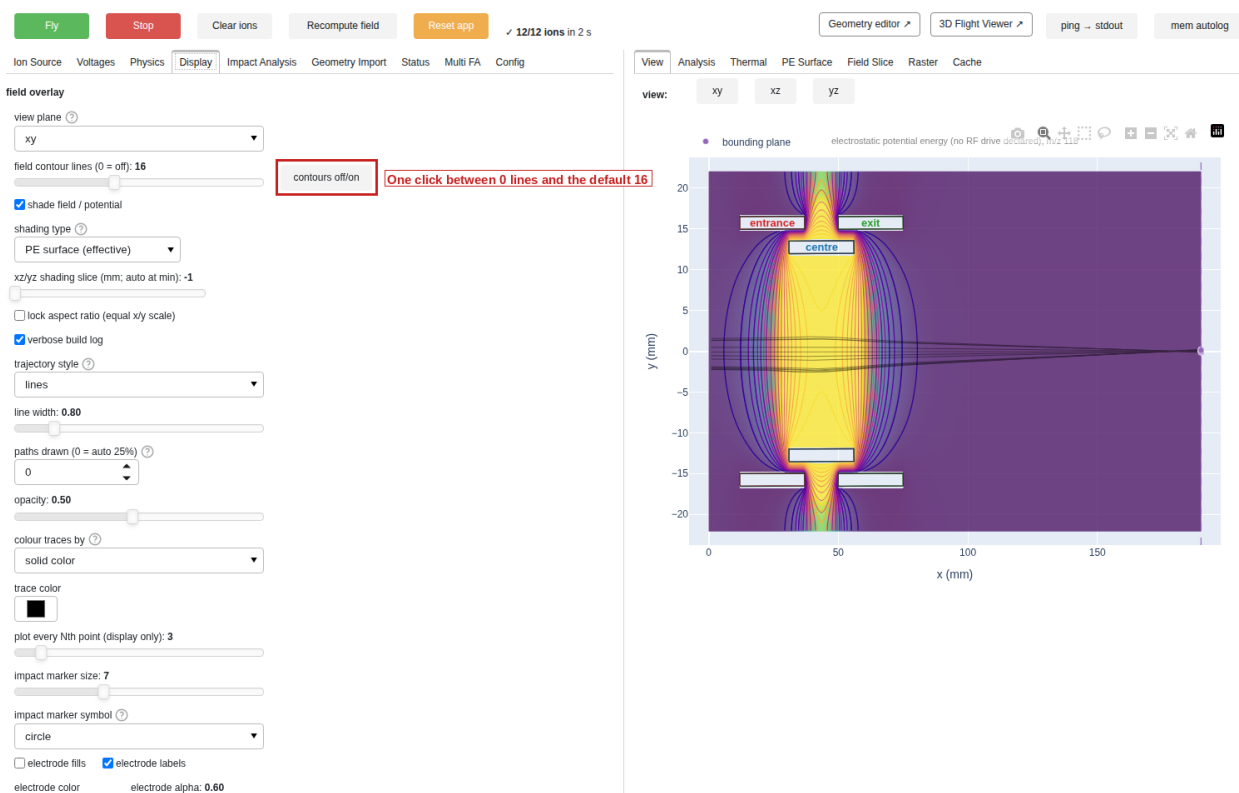
Step 3 -- check the flight conditions. The Physics tab's Integration sub-tab carries the conditions Einzel Lens (r-z) ships with: a 2 ns time step and a 60 μ s flight-time ceiling. Collisions live on the Gas sub-tab beside it and are off for this deck (vacuum). What these fields show is what the integrator receives.

Step 4 — press Fly. The solve runs first (about ten seconds for this geometry; its progress prints to the Status tab), the ions fly, and the trajectories draw over the solved geometry in the View pane; xy / xz / yz projections are one click apart, and the 3D Flight Viewer opens the same paths in a rotatable scene.



Step 4 -- press Fly: the trajectories draw over the solved geometry in the View pane (the one-button form of the section 2.1 script). xy / xz / yz projections are one click apart, and the 3D Flight Viewer opens the same paths in a rotatable scene.

Step 5 — show the field. On the Display tab, tick “shade field / potential”: the pane shades the field the ions actually flew through, with contour lines at the default count of 16. The contours off/on button beside the slider flips the lines with one click; the slider is the single value authority.



Step 5 -- the potential-energy landscape the ions flew through: Display tab, shading type 'PE surface (effective)', contour lines at the default 16. For this DC lens the PE surface is the electrostatic potential energy of the ion, so the einzel's three-tube focusing structure reads directly as the shape of the well; the contours off/on button flips the lines with one click.

Step 6 — read the result. The Status tab's run log names the solve, the fly, and the per-fate counts — the ground truth for what the flight did. For this deck expect all 12 ions to complete, every one ending on the bounding plane past the lens (the beam is focused through and exits the domain), with a median time of flight near 17 μ s. The Analysis tab plots any recorded channel against any other for the same run (press Plot after choosing).

Fly

Stop

Clear ions

Recompute field

Reset app

✓ 12/12 ions in 2 s

Ion Source

Voltages

Physics

Display

Impact Analysis

Geometry Import

Status

Multi FA

Config

current

done: 12 ions in 0.1 s (12 bounding plane) · median tof 16.87 μ s · stored as 'einzell lens (r-z, three-tube)' — Abdulla & Hussein 2020 geometry, decelerating mode [00:07:05] (reloadable) · /flight #1

workers

worker threads

1

1 thread(s) · 0 in flight · 0 queued · 12/12 done · 159.7 ions/s

history (this session, newest first, last 100)

00:07:06

done: 12 ions in 0.1 s (12 bounding plane) · median tof 16.87 μ s · stored as 'einzell lens (r-z, three-tube)' — Abdulla & Hussein 2020 geometry, decelerating mode [00:07:05] (reloadable) · /flight #1

00:07:05

12/12 ions — 160/s

00:07:03

flying 12 ions... (a first fly in a fresh session pauses ~30-60 s compiling the tracer before ion 1 — Stop takes effect after that)

00:07:02

loading cached field... 0s elapsed (one-time for this geometry; voltage & view changes after this are instant)

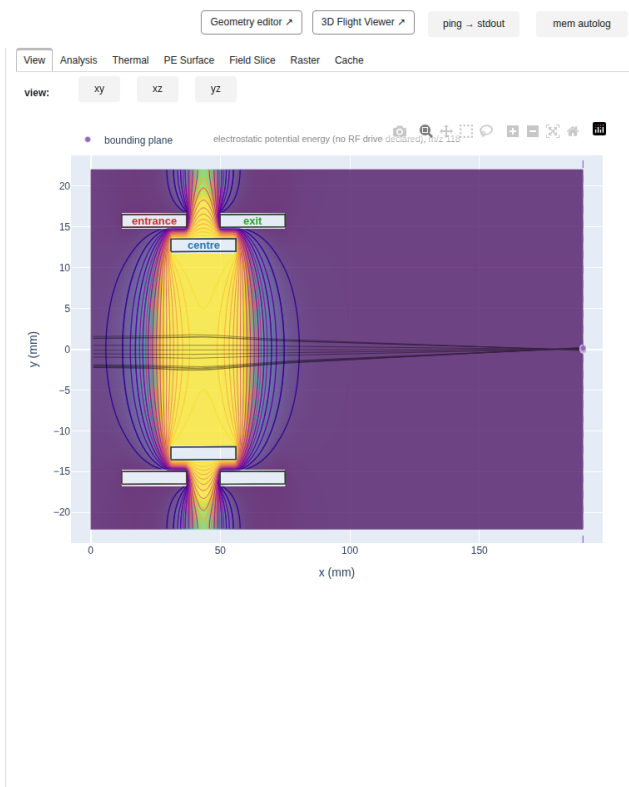
00:06:51

loaded: einzel lens (r-z, three-tube) — Abdulla & Hussein 2020 geometry, decelerating mode — press Recompute field to solve, or Fly to run.

00:06:49

loading einzel lens (r-z, three-tube) — Abdulla & Hussein 2020 geometry, decelerating mode... rendering geometry (3 electrodes) — large models can take ~10-30 s.

00:06:46



Step 6 -- the expected result, in the Status tab's run log: the solve, the fly, and the per-fate counts for the 12 ions. This log is the ground truth the chapter-2 prose quotes.

3. The specification file

A specification is JSON with a small number of top-level sections. The sections that matter most are:

Section	What it declares
geometry	Grid pitch, extents, symmetry, electrodes, their voltages and RF group membership, and how the field is differenced.
source	How many ions, their m/z , their spatial and velocity distribution, and the random seed.
integration	Time step, maximum flight time, and how often trajectory points are recorded.
collisions	Background gas species, pressure and temperature, or none for vacuum.
bounds	Optional planes that terminate a flight when crossed.

Two properties of this file are worth internalizing. First, **the resolution is declared, not inferred**: `mm_per_gu` is the grid pitch in millimeters, and it sets both the accuracy and the cost of every solve. Second, **defaults are omitted**. A field left out of the file takes its documented default and is not written back, so two specifications that differ only in a default are byte-identical files.

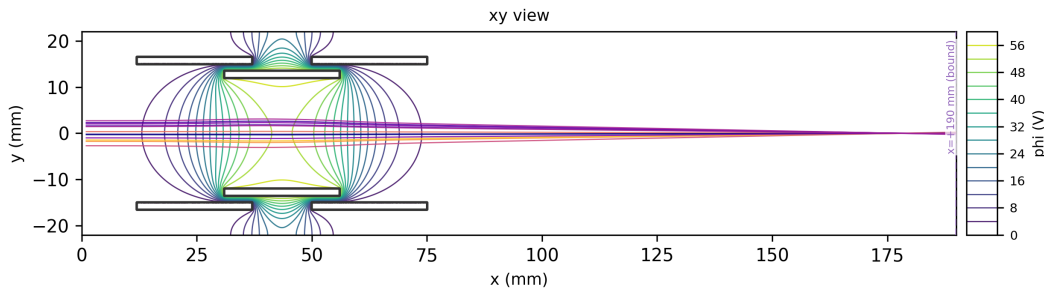
4. Geometry

4.1 Declared primitives

The simplest geometries are built from declared shapes — boxes, cylinders, and their unions and differences — attached to named electrodes. Each electrode carries a name, an index, a DC voltage, and optionally the name of an RF group it belongs to. The name is not decoration: it is what the field maps, the legends and the analysis channels refer to.

einzel lens (r-z, three-tube) — Abdulla & Hussein 2020 geometry, decelerating mode [SOLVER-DERIVED (geometry + field as flown)]

fill = DC (0 to 57 V)



- domain 190 x 44 mm, h = 0.1 mm/gu (node-centred: node (0,0) at 0, -22 mm)
- 3 electrode(s); metal drawn from the solver's own mask
- 12 ion path(s) overlaid, as flown
- 1 declared bounding plane(s) drawn dashed
- 12 ion(s) crossed a declared plane

Figure 1. A round einzel lens declared as cylindrical (r-z) primitives. The shading is the solved potential; the outlined bodies are the electrodes exactly as the solver received them. The radial axis is drawn from $-R$ to $+R$ with the symmetry axis at zero.

4.2 Importing STL meshes

When a shape cannot be expressed as primitives, it is imported as an STL mesh — one file per electrode. This is the escape hatch for real hardware: draw the part in SolidWorks, Onshape or CadQuery, export STL, and point the specification at it.

The specification declares a directory with `stl_dir`, and each electrode names its own file. The directory is resolved relative to the specification file, so a specification and its meshes move together.

```
"geometry": {
  "mm_per_gu": 0.2,
  "stl_dir": "einzel_stl_plates",
  "electrodes": [
    {"index": 1, "name": "plate_1", "dc": 0.0, "stl": "einzel3d_plate_1.stl"},
    {"index": 2, "name": "plate_2", "dc": -2000.0, "stl": "einzel3d_plate_2.stl"},
    {"index": 3, "name": "plate_3", "dc": 0.0, "stl": "einzel3d_plate_3.stl"}
  ]
}
```

Meshes are voxelized onto the same uniform grid as everything else. A mesh does not buy you sub-grid accuracy: a curved surface is represented by the grid cells it occupies, so the solver pitch still governs how faithfully a curve or a thin feature is captured. A feature thinner than about two grid cells is not reliably resolved, and a mesh with holes or flipped normals may voxelize to something that is not what you drew. There is a mesh feature which will be described later.

Because the geometry is only as trustworthy as the files behind it, the toolkit records a SHA-256 hash of every STL a specification references. The preflight check compares the files on disk against those hashes and refuses to proceed on a mismatch, rather than silently solving a geometry that is no longer the one the specification describes.

quadrupole — STL rods, full 3-D (120 mm) [SOLVER-DERIVED (geometry + field as flown)]

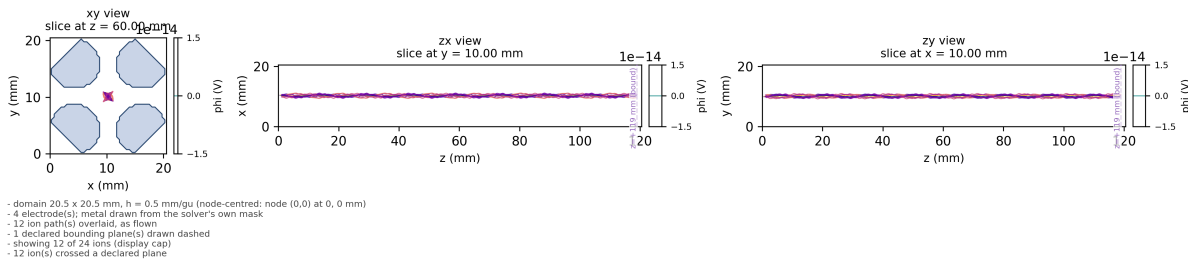


Figure 2. A quadrupole imported from STL rod meshes and solved as a full 3-D geometry, shown in three orthogonal projections. Any geometry with genuine depth is presented multi-axis: a single projection hides exactly the off-axis structure one needs to see.

4.3 File organization

A simulation is a directory, not a single file. The convention is:

```
my_instrument/
  instrument.json      <- the specification
  stl/                 <- meshes named by electrode
    plate_1.stl
    plate_2.stl
  births.csv           <- optional explicit ion start conditions
```

Keep the specification at the root of its own directory and reference resources with relative paths. Absolute paths make a specification unusable on any other machine, and are the single most common reason a shared simulation fails to load. Solved fields are cached separately, outside the specification directory, and are regenerated on demand — they are not part of what you share.

While projects folders with multiple devices (i.e., and instrument), STLs, and custom ion birth options may become more complex, **a single json file is all that is needed to reproduce a project result for a single set of electrodes.**

4.4 Symmetry and mirror planes

If a geometry is symmetric about a plane, only half of it need be stored and solved. A declared mirror axis halves the memory and roughly halves the solve time, and the solver reflects the appropriate boundary condition at that plane.

Symmetry is **declared, never assumed**. The toolkit will not detect symmetry for you, because a geometry that looks symmetric may not be symmetric in its voltages (e.g. a hexapole), and solving the wrong half is a silent error. A mirrored axis is displayed from $-H$ to $+H$ with the mirror plane at zero, so a coordinate you write in the specification is measured from the plane of symmetry and read back the same way.

5. Solving the field

Why a grid at all? Between the electrodes the potential obeys Laplace's equation, and for any realistic electrode shape that boundary-value problem has no closed-form solution — the geometry IS the problem. The classical response is to replace the continuous domain with a regular lattice of sample points, write the equation as a relation between each point and its neighbors (each node relaxes toward the average of its neighbors — the discrete statement that a harmonic function has no interior extrema), and iterate until the lattice stops changing. Everything downstream inherits the choice: the pitch of that lattice is the resolution of the physics, which is why the specification declares it explicitly instead of inferring it.

5.1 Per-electrode bases and superposition

The field is not solved once for a set of voltages. Instead, Laplace's equation is solved once per electrode, with that electrode at a reference potential and every other electrode grounded. Each solution is stored normalized to unit electrode potential, giving a dimensionless response function — the fraction of that electrode's voltage seen at each point.

The field for any set of voltages is then the weighted sum of these bases. This has a consequence worth exploiting: changing a voltage is not a re-solve. Sweeping a lens voltage, retuning a stack, or optimizing a set of potentials re-weights bases that are already in memory, which is essentially free. Changing the geometry or the grid pitch is a re-solve, because the bases themselves depend on those.

Because a solve is expensive and a geometry is often reused, solved bases are cached on disk under a key derived from everything that affects them — the geometry, the pitch, the declared symmetry, and the mesh hashes. Any change to those produces a different key and a fresh solve.

5.2 The iterative solvers

Laplace's equation is discretized on the uniform grid and relaxed iteratively. The base method is successive over-relaxation (SOR), the classical accelerated relaxation scheme analyzed by Young [3], applied in a red-black ordering so that each half-sweep is independent and can be executed in parallel. The relaxation factor is chosen from the grid dimensions rather than left to the user.

SOR alone converges slowly on large grids because it removes short-wavelength error (components of the residual that vary over a few grid cells) quickly and long-wavelength error (components that vary over a large fraction of the domain) very slowly. The multigrid solver addresses this by relaxing on a hierarchy of coarser grids, where the long-wavelength error is short-wavelength, and prolongating the result upward — the multi-level idea introduced by Brandt [2]. The implementation here is cascadic: it solves coarse-to-fine, using each level's solution as the next level's starting guess, and the final answer is produced by relaxing on the finest grid to the requested tolerance. This matters for a subtle reason — the fine-grid solution is the one the ions fly through, so it is solved directly rather than reconstructed from a correction.

Convergence is tested against a declared tolerance on the maximum change per sweep (watch the terminal console during a large solve to see this in action). If the solver reaches its iteration cap without meeting the tolerance it raises an error. It does not return the unconverged field. This is deliberate: an unconverged field that looks plausible is far more dangerous than a failed solve, because nothing downstream can detect it.

6. From potential to field

The solver produces a potential. The trajectory kernel needs an electric field, $E = -\nabla\phi$, evaluated on the same grid. How that gradient is taken is a choice with real consequences, and it is a declared option.

6.1 The two methods

Plain gradient takes a central difference everywhere: the field at a node is computed from its two neighbors along each axis. This is second-order accurate and is the correct thing to do anywhere in the vacuum between electrodes.

Electrode-aware differs only at metal surfaces. At a node that lies on a conductor and has vacuum on exactly one side along an axis, the central difference straddles the electrode — and because the potential inside a conductor is flat, the half of the stencil inside the metal contributes nothing. The result is a surface field roughly half its true value. The electrode-aware method replaces that one derivative with a one-sided difference taken into the vacuum, never across the metal interior. Vacuum nodes are untouched, so the two methods differ only on the one-cell skin where the metal faces the gap.

6.2 When each is appropriate

Situation	Use	Why
Ions born on an electrode surface	electrode-aware	The ion's first acceleration happens in exactly the cells where the plain gradient is wrong. A surface-born ion is mis-accelerated by roughly one percent of the gap energy without the correction — small, but systematic, and it maps directly onto arrival-time error.
Ions grazing or striking surfaces	electrode-aware	Whenever the trajectory samples the surface skin, the surface field is part of the answer.
Transport through a central gap	plain gradient	If ions stay away from the electrodes — the usual case for a guide, a funnel, or a transport stack — the correction touches cells the ions never visit. It is then pure cost.
Large grids where speed matters	plain gradient	The correction is a per-metal-node pass, so its cost scales with the electrode surface area on the grid.

The default is electrode-aware, because being right at surfaces is the safer default and the situations that need it are the ones where being wrong is least visible. Choose the plain gradient deliberately, when you can say why the ions do not sample the surfaces.

A related option controls the **stored precision** of the drive-field arrays. The gradient is always computed in double precision; the option decides whether the result is stored as 32-bit or 64-bit. Single precision halves the memory of the field arrays, which becomes the binding constraint on large three-dimensional grids, at a rounding-level cost that is negligible against the discretization error already present. It is stated as an option rather than chosen for you because a frozen reference result stored at one precision will not reproduce bit-for-bit at the other.

7. Flying ions

7.1 The integrator

Trajectories are integrated with the classical fourth-order Runge-Kutta method, the same scheme commonly used for trajectory integration [1]. The field is sampled at the ion's position by trilinear interpolation between the eight surrounding grid nodes, which is what makes the grid pitch the dominant accuracy parameter: the interpolated field is smooth, but it is smooth between samples of a field that was itself computed on that grid.

The time step is declared rather than adapted. This is a current design choice that could be modified in the future, though the present approach is clean and valid. The practical consequence is that the step must be checked against the fastest motion in the problem — for an RF device, against the RF period, not against the transit time. A step that comfortably resolves the drift will silently misrepresent the micromotion.

The reason is worth spelling out. A Runge-Kutta step assumes the force varies smoothly ACROSS the step: its error per step grows steeply with how much the field changes within one Δt . Motion in a 3D RF trap or a tight lens is oscillatory, and sampling an oscillation with too few points per cycle does not merely blur it — it aliases it, producing a trajectory that is systematically wrong (spurious heating or damping, a shifted secular frequency) while looking perfectly smooth. That is why the recommend rule is ~ 20 steps per RF period, counted against the FASTEST drive present, and why a vacuum TOF drift tolerates a far coarser step than the same ion inside an RF device.

Convergence in Δt is checked the same way convergence is always checked: halve the step, re-fly, and compare the observable you actually care about (a time-of-flight, a transmitted fraction, a spot size). If the answer moves, the first step was too coarse; if it does not, you have measured — not assumed — that the step is converged. The toolkit will not do this for you, deliberately: which observable must be converged is a physics judgment, not a solver setting. However, there are recommendations returned to the user in the UI that estimate the time step needed.

7.2 Time-varying drives

Electrodes are grouped, and each group carries a waveform: sine, cosine, square with an adjustable duty cycle, or an arbitrary tabulated shape. The field seen by an ion is the static field plus the sum of each group's field weighted by its waveform at that instant. This is a full time-domain treatment — the RF is integrated as it actually varies, not approximated — and it is what makes traveling-wave devices expressible: a traveling wave is a set of groups with a phase progression.

7.3 Collisions with background gas

Two gas models are available, and they occupy opposite ends of the pressure axis. HS treats collisions as discrete events: between collisions the ion flies the electric field unperturbed; at each step a collision probability is computed from the hard-sphere cross-section, the gas density and the relative speed, and when a collision fires, a gas atom is drawn from the thermal distribution and the pair scatter elastically. Every kick is individually resolved, which is exactly right when collisions are sparse — a collision cell at tens of millitorr, a funnel entrance — and increasingly expensive and unnecessary as pressure rises and thousands of collisions occur per RF cycle.

When a background gas is declared, ion-neutral collisions are modeled with the Statistical Diffusion Simulation of Appelhans and Dahl [4]. Rather than tracking individual collisions, SDS applies two effects per time step: a viscous damping of the acceleration toward the local mobility-limited drift velocity, and a random-walk displacement drawn from tabulated diffusion statistics scaled by the local mean free path. This is a faithful port of the published model, and its assumptions are inherited with it — in particular it is a statistical treatment valid when many collisions occur per time step, not a single-collision model.

The choice between them is therefore a physics statement, not a preference: HS where individual collisions are the phenomenon (energy relaxation, collision counting, low pressure), SDS where the gas acts as a continuum drag-and-diffusion bath (mobility-dominated transport at roughly a Torr and above). Running SDS at a pressure so low that few collisions occur per step violates its founding assumption, and running HS deep into the mobility regime is paying per-collision cost for a statistically determined answer.

7.4 Units

One unit convention runs through the whole toolkit. Mixing these up is the most common source of results that are wrong by exactly a factor of a thousand:

Quantity	Unit
Length	millimeters (mm)
Time	microseconds (μs)
Velocity	mm/ μs (numerically equal to km/s)
Potential	volts (V)
Electric field	volts per millimeter (V/mm)
Energy	electronvolts (eV)
Mass	atomic mass units, as m/z

Quantity	Unit
Pressure / temperature	Torr / kelvin

8. The pseudopotential view

An RF field exerts a net focusing force on an ion because the field is inhomogeneous: over one RF cycle the ion spends slightly longer where the field is weaker. Averaged over the cycle this is equivalent to a static effective potential — the pseudopotential of Dehmelt [5] — of magnitude

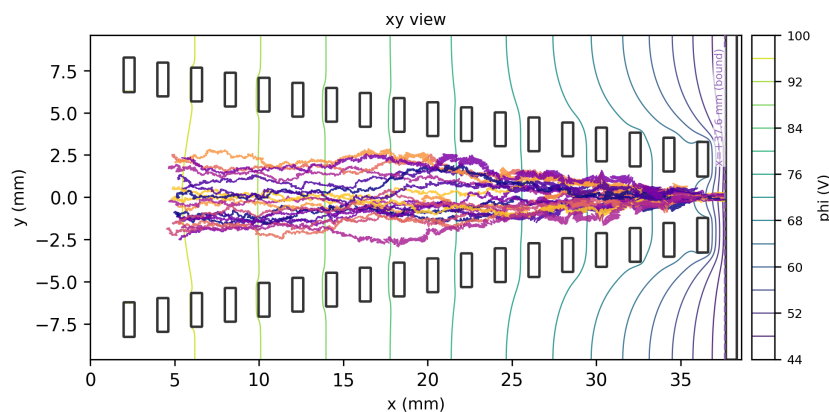
$$V^* = q E_0^2 / (4 m \Omega^2)$$

where E_0 is the amplitude of the RF field at that point, m the ion mass, and Ω the angular drive frequency. The toolkit displays the sum of the DC potential energy and this term as an effective potential-energy surface, which is the natural way to see where ions are confined and how deep the well is.

This picture is an approximation and the flight is not. Trajectories are always integrated in the real time-varying field; the pseudopotential is a visualization. It is valid in the adiabatic limit, where the field changes little over one RF period as seen by the ion — the condition quantified by the adiabaticity parameter of Gerlich [6]. It degrades where the field is strongly inhomogeneous over a micromotion amplitude, near electrode surfaces, at low m/z , and near the edges of stability. When the pseudopotential says an ion is confined and the trajectory says it is lost, the trajectory is the answer.

ion funnel — 18-ring taper + 45 V exit plate [SOLVER-DERIVED (geometry + field as flown)]

fill = DC (45 to 100 V)



- domain 38.6 x 19.2 mm, h = 0.05 mm/gu (node-centred: node (0,0) at 0, -9.6 mm)
- 19 electrode(s); metal drawn from the solver's own mask
- 20 ion path(s) overlaid, as flown
- 1 declared bounding plane(s) drawn dashed
- showing 20 of 75 ions (display cap)
- 20 ion(s) crossed a declared plane

Figure 3. An electrodynamic ion funnel [7]: a stack of ring electrodes of decreasing aperture, driven with RF on alternating rings and a DC gradient along the axis. The RF confines ions radially through the pseudopotential while the DC pushes them forward; the geometry is the solver's own electrode mask, not a schematic.

9. Reading results

A flight returns a trajectory array and a summary. The trajectory array has one row per recorded step; the column names come back from the build, so read them rather than assuming an order. Recorded per step are time, position, velocity, and the electric field components the ion actually experienced at that point. From these the interface derives further channels — speed, kinetic energy in eV, radial position, field magnitude — which can be plotted against one another for a single ion or a whole population.

The summary reports how the flight ended, and that outcome deserves attention: an ion that timed out is not the same as one that struck an electrode or left through a declared boundary, and a population statistic computed without separating those is usually meaningless.

Recording is subsampled by a declared interval. This decision is adjustable by the user and an option to save memory. Additionally, every recorded quantity carries its unit in the interface, and the units are those of section 7.4 — in particular the field channels are in V/mm, not V/m.

10. Limitations

The follow are properties of the ion_gym approach, not defects. Knowing them is the difference between a simulation you can defend and one you cannot.

- **Electrostatics only.** Laplace's equation with conductors at fixed potentials. No magnetic fields. No dielectric materials. No space charge, so the results are single-ion results: they do not describe a beam dense enough for ion-ion repulsion to matter, and they will not reproduce space-charge limits on transmission or trapping capacity.
- **A uniform grid.** Everything is discretized at one pitch. Accuracy is governed by that pitch, and geometric detail below roughly two cells is not resolved. There is no local refinement, so a fine feature inside a large domain forces a fine pitch everywhere and the cost grows as the cube of the refinement in three dimensions.
- **Voxelized surfaces.** A curved or angled electrode becomes a staircase of grid cells. This matters most where the surface field matters most — near sharp features and in narrow gaps — and it is a discretization error that no choice of gradient method removes.
- **A fixed time step.** The integrator does not adapt. The error of a Runge-Kutta step grows steeply with how much the force changes within the step, so the step must be chosen against the fastest motion present — the RF period in a driven device, the sharpest field gradient in a static one — not against the transit time. An adaptive integrator would detect a too-coarse step and shrink it; this one will integrate an undersampled oscillation into a smooth, plausible, wrong trajectory. Verifying convergence is therefore the user's responsibility, and the check is mechanical: halve Δt , re-fly, and confirm the observable you are reporting does not move (section 7.1).
- **Statistical collisions.** The gas model is a diffusion-and-mobility treatment valid in the many-collisions-per-step regime. It has no flow field, so it cannot represent jets, expansions, or pressure gradients across a device; a step change in pressure between regions is an idealization of a differential-pumping aperture, not a simulation of one.
- **The pseudopotential is a picture.** As described in section 8 it is an adiabatic approximation used for display. Its RF term is built from the field gradient within the displayed plane, so on a slice through a structure that varies steeply out of that plane it understates the true confinement.
- **Symmetry is a declaration.** A mirror plane is applied because the specification asserts it. Asserting a symmetry the voltages do not actually have produces a field that is wrong in a way the solver cannot detect.

11. Dashboard reference (tab by tab)

Every control below is documented from the application itself: the UI harvester walks the live widget tree and emits this section from the widgets' own description strings, so the manual API simulation cannot drift from the app. Hover any control in the dashboard for the same text.

Ion Source

FlyStopClear ionsRecompute fieldReset app

Ion Source Voltages Physics Display Impact Analysis Geometry Import Status Multi FA Config

ions per m/z 25

ions flown PER m/z -- total = this x number of masses

m/z list (Da, comma-separated)
118, 322, 622

One block of ions per listed mass

Each m/z is flown with its OWN block of ions per m/z ions -- e.g. 50 with 3 masses flies 150, 50 of each. These govern Basic and Advanced.

BasicAdvancedStationsStats

birth distribution box

Spatial pattern of the birth cloud

start x (mm) 5

start y (mm) 0

start z (mm) 0

box dx (mm) 1

box dy (mm) 5

box dz (mm) 1

KE min (eV) 0.1

KE max (eV)

temperature (K)

Birth kinetic energy drawn uniformly in [min, max]

launch direction (KE is along this vector):

launch dir x 1

launch dir y

launch dir z

Birth KE is directed along this vector (normalized)

Clear ions

Geometry editor3D Flight Viewerping -> stdoutmem autolog

ViewAnalysisThermalPE SurfaceField SliceRasterCache

view: xyxzyz

Ion Source tab, box birth distribution (the default): ions are placed uniformly inside a rectangular volume whose extents are the box dx/dy/dz fields.

[Fly](#)
[Stop](#)
[Clear ions](#)
[Recompute field](#)
[Reset app](#)

[Ion Source](#)
[Voltages](#)
[Physics](#)
[Display](#)
[Impact Analysis](#)
[Geometry Import](#)
[Status](#)
[Multi FA](#)
[Config](#)

ions per m/z
☐ seeded run
 seed

m/z list (Da, comma-separated)

Each m/z is flown with its OWN block of ions per m/z ions — e.g. 50 with 3 masses files 150, 50 of each. These govern Basic and Advanced.

[Basic](#)
[Advanced](#)
[Stations](#)
[Stats](#)

birth distribution
point: a single birth position

start x (mm)
 start y (mm)
 start z (mm)
Cloud centre; in r-z geometry x is the axis

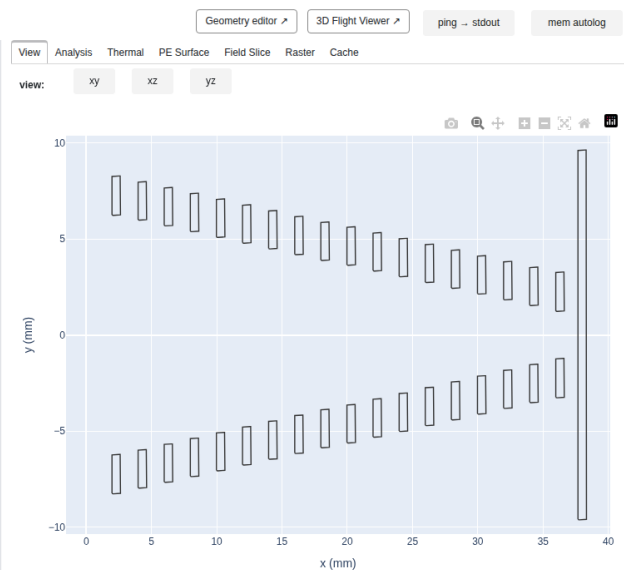
point source — all ions born exactly at the centre (no size parameters)

KE min (eV)
 KE max (eV)
 temperature (K)

launch direction (KE is along this vector):

launch dir x
 launch dir y
 launch dir z

[Clear ions](#)



Ion Source tab, point birth distribution: zero declared extent, so downstream spread is set entirely by the optics (the app prints a point-source advisory for exactly this case).

[Fly](#)
[Stop](#)
[Clear ions](#)
[Recompute field](#)
[Reset app](#)

[Ion Source](#)
[Voltages](#)
[Physics](#)
[Display](#)
[Impact Analysis](#)
[Geometry Import](#)
[Status](#)
[Multi FA](#)
[Config](#)

ions per m/z
☐ seeded run
 seed

m/z list (Da, comma-separated)

Each m/z is flown with its OWN block of ions per m/z ions — e.g. 50 with 3 masses files 150, 50 of each. These govern Basic and Advanced.

[Basic](#)
[Advanced](#)
[Stations](#)
[Stats](#)

birth distribution
disc: a filled circle

start x (mm)
 start y (mm)
 start z (mm)

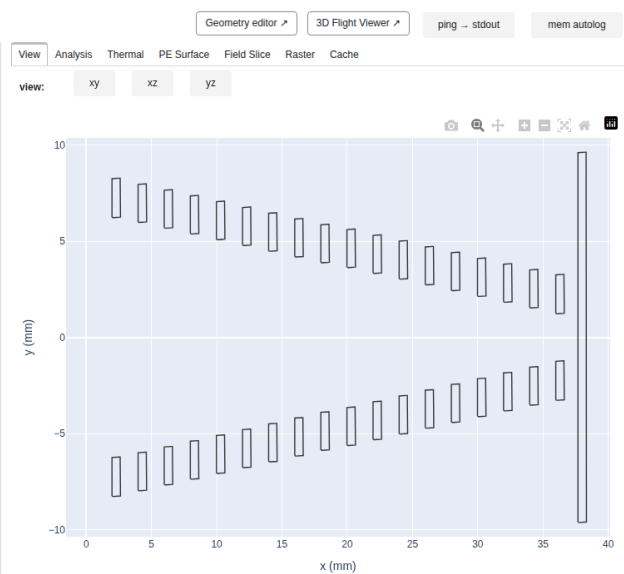
birth cloud size (mm)
Disc radius of the birth cloud

KE min (eV)
 KE max (eV)
 temperature (K)

launch direction (KE is along this vector):

launch dir x
 launch dir y
 launch dir z

[Clear ions](#)



Ion Source tab, disc birth distribution: a filled disc of the given radius around the start position.

[Fly](#)
[Stop](#)
[Clear ions](#)
[Recompute field](#)
[Reset app](#)

[Ion Source](#)
[Voltages](#)
[Physics](#)
[Display](#)
[Impact Analysis](#)
[Geometry Import](#)
[Status](#)
[Multi FA](#)
[Config](#)

ions per m/z
☐ seeded run
 seed

m/z list (Da, comma-separated)

Each m/z is flown with its OWN block of ions per m/z ions — e.g. 50 with 3 masses files 150, 50 of each. These govern Basic and Advanced.

[Basic](#)
[Advanced](#)
[Stations](#)
[Stats](#)

birth distribution

start x (mm)
 start y (mm)
 start z (mm)

FWHM x (mm)
 FWHM y (mm)
 FWHM z (mm)
Gaussian FWHM per axis; 0 = no spread on that axis

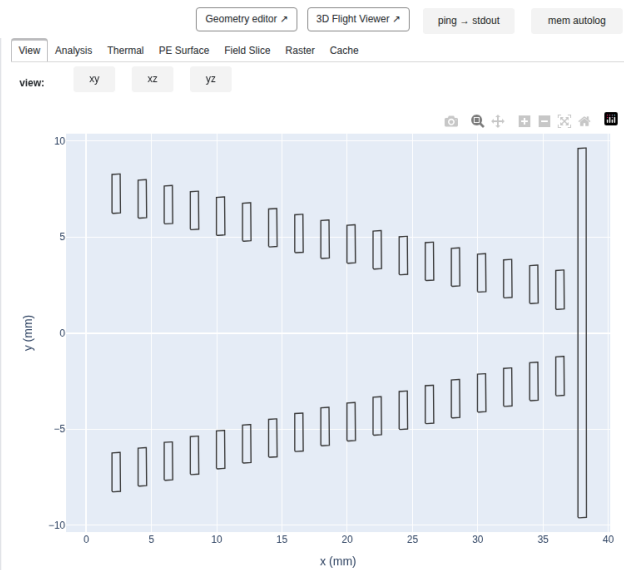
wings kept to $\pm 3\sigma$ automatically; custom truncation is a deck/API setting (source.trunc_mm), not a UI control

KE min (eV)
 KE max (eV)
 temperature (K)

launch direction (KE is along this vector):

launch dir x
 launch dir y
 launch dir z

[Clear ions](#)



Ion Source tab, gaussian birth distribution: per-axis FWHM; truncation is derived at 3 sigma -- an untruncated Gaussian is a different beam, and the derived cut removes the wings without another knob.

[Fly](#)
[Stop](#)
[Clear ions](#)
[Recompute field](#)
[Reset app](#)

[Ion Source](#)
[Voltages](#)
[Physics](#)
[Display](#)
[Impact Analysis](#)
[Geometry Import](#)
[Status](#)
[Multi FA](#)
[Config](#)

ions per m/z
☐ seeded run
 seed

m/z list (Da, comma-separated)

Each m/z is flown with its OWN block of ions per m/z ions — e.g. 50 with 3 masses files 150, 50 of each. These govern Basic and Advanced.

[Basic](#)
[Advanced](#)
[Stations](#)
[Stats](#)

Detector / monitor stations — a plane at **axis** = **pos** with optional windows over the other axes, **detect** absorbs at first in-window crossing; **record** logs and lets the ion continue.

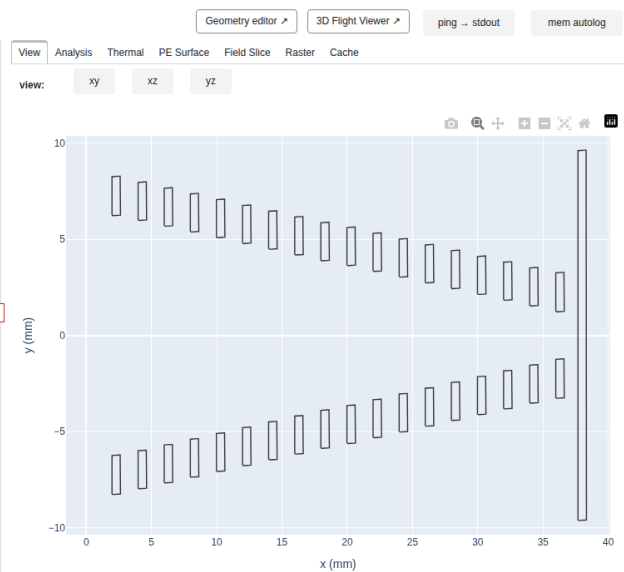
station
 name
 kind
detect absorbs; record logs and continues

axis
 pos (mm)
Plane position on the axis

☐ window y
 min
 max

☐ window z
 min
 max

[Apply station](#)
[Delete](#)
Writes into the live spec and the JSON box



The station editor (Ion Source > Stations): a detector or monitor plane at axis = pos with optional windows over the other two axes. 'detect' absorbs at first in-window crossing; 'record' logs and lets the ion continue. Apply station writes into the live specification and the JSON box.

- **ions per m/z (IntInput)** — Ions flown PER m/z value. Total flown = this x the number of m/z entries. E.g. 50 with 4 masses flies 200 ions (50 of each).
- **seeded run (Checkbox)**
- **seed (IntInput)** — Pins the whole run: births, birth times and collision draws. Ion i always draws the same numbers, so reruns are bit-identical and two tunes are compared on common random numbers. Unchecked (default): fresh entropy per run, drawn seed printed.
- **m/z list (Da, comma-separated) (TextInput)**
- **birth distribution (Select)** — Spatial pattern the ions are seeded over: a single point, a filled disc, a line, a grid, or a BOX (random uniform placement inside a rectangular volume -- avoids the skew of a contrived single-point source; extents set below).
- **start x (mm) (FloatInput)** — Birth position of the ion cloud center, x (mm). In r - z geometry x is the axis.
- **start y (mm) (FloatInput)** — Birth position of the ion cloud center, y (mm). In r - z geometry y is the radius.
- **start z (mm) (FloatInput)** — Birth position z (mm) — relevant for 3-D scenes.
- **box dx (mm) (FloatInput)** — Full extent (mm) of the birth box along this axis, centered on (start x , y , z); each ion is placed uniformly at random inside. 0 collapses that axis.
- **box dy (mm) (FloatInput)** — Full extent (mm) of the birth box along this axis, centered on (start x , y , z); each ion is placed uniformly at random inside. 0 collapses that axis.
- **box dz (mm) (FloatInput)** — Full extent (mm) of the birth box along this axis, centered on (start x , y , z); each ion is placed uniformly at random inside. 0 collapses that axis.
- **KE min (eV) (FloatInput)** — Lower bound of the birth kinetic energy; each ion draws uniformly in [min, max]. Set equal for monoenergetic.
- **KE max (eV) (FloatInput)** — Upper bound of the birth kinetic energy.
- **temperature (K) (FloatInput)** — Thermal velocity spread SUPERPOSED on the directed beam KE (a beam AT a temperature; needs $\geq v329$ semantics). 0 = off. This is the turn-around-time knob in a TOF source.

launch direction** (KE is along this vector):

- **launch dir x (FloatInput)** — Launch direction vector (the birth KE is directed along this; it is normalized, so only the direction matters, not the magnitude). e.g. (0,0,1) fires along z .
- **launch dir y (FloatInput)** — Launch direction vector (the birth KE is directed along this; it is normalized, so only the direction matters, not the magnitude). e.g. (0,0,1) fires along z .
- **launch dir z (FloatInput)** — Launch direction vector (the birth KE is directed along this; it is normalized, so only the direction matters, not the magnitude). e.g. (0,0,1) fires along z .
- **Clear ions (Button)** — Clear the flown ions from the view (same as the top-bar Clear ions).

Beam declaration

Below the birth-cloud controls sits the beam declaration: a compact statement of the packet as a beam. It fixes the ion count and seed, names the propagation axis, and then gives one row per phase-space channel — the two transverse positions, the three direction components, the kinetic energy and the birth time. Every row has the same shape: a distribution chooser and its one or two parameters, so declaring a beam reads as a table of channels rather than a scatter of boxes. Regenerate beam draws a fresh packet from the declaration; Derive from current packet runs the other direction, fitting the declaration to whatever packet is currently loaded — the usual way to turn a flown arrival distribution into the birth condition of the next stage.

The station editor follows (Ion Source > Stations): a detector or monitor plane at axis = pos with optional windows over the other two axes. Kind `detect` absorbs the ion at its first in-window crossing; `record` logs the crossing and lets it continue. Apply station writes into the live specification and the JSON box, so a save persists it. Last, the Stats sub-tab selectors choose which population the statistics

table describes: which surface, which transmitted fate, and which plane the impact cross-section is taken in.

Voltages

Fly

Stop

Clear ions

Recompute field

Reset app

Ion Source | Voltages | Physics | Display | Impact Analysis | Geometry Import | Status | Multi FA | Config

Load drive template (voltage groups only, from another json):

Choose File | No file chosen

load a drive template: pulls rf + dc groups from another json, clears electrode assignments for re-binding

Drive groups (define waveform + amplitude/frequency/phase once; assign electrodes below):

RFA waveform (sin) | RFA amplitude (V) (75) | RFA frequency (Hz) (850000) | RFA phase (deg) (Per-group drive rate) | RFA duty (0.5) | RFA pulse delay t (μs) (1)

RFA — members (e1 — ring_01 | e3 — ring_03 | e5 — ring_05 | e7 — ring_07 | e9 — ring_09 | e11 — ring_11 | e13 — ring_13 | e15 — ring_15 | e17 — ring_17)

RFB waveform (sin) | RFB amplitude (V) (75) | RFB frequency (Hz) (850000) | RFB phase (deg) (180) | RFB duty (0.5) | RFB pulse delay t (μs) (1)

RFB — members (e2 — ring_02 | e4 — ring_04 | e6 — ring_06 | e8 — ring_08 | e10 — ring_10 | e12 — ring_12 | e14 — ring_14 | e16 — ring_16 | e18 — ring_18)

new group name (e.g. RF3) | waveform (sin) | + add drive group | remove group(s)

Geometry editor | 3D Flight Viewer | ping -> stdout | mem autolog

View | Analysis | Thermal | PE Surface | Field Slice | Raster | Cache

view: xy | xz | yz

Voltages tab, drive groups: waveform, amplitude, frequency, phase and members are defined once per group; electrodes are assigned below.

new group name:
 waveform:

travelling-wave builder

TW ladder electrodes (in order)

phases (N)
 waveform
 TW freq (kHz)
 amp (V)
 offset (V)
 prefix

DC groups — feed a chain at both ends; members are interpolated by their number.

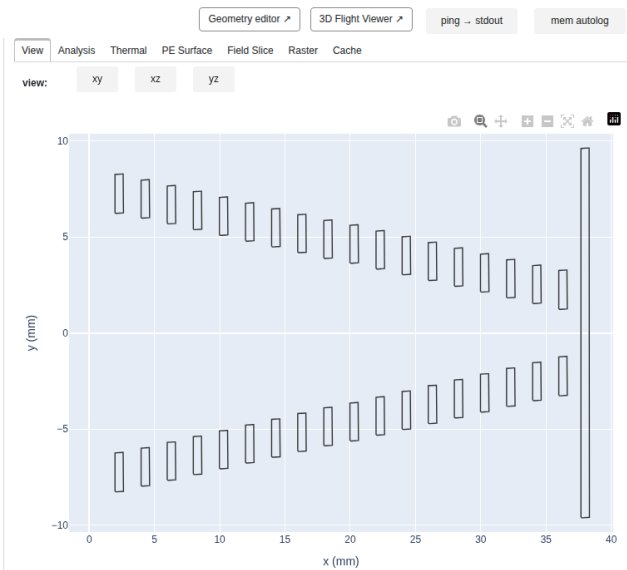
funnel_ladder — DC in (V)
 funnel_ladder — DC out (V)
Applied to the LOWEST-numbered member

funnel_ladder — members

e1 — ring_01 x	e2 — ring_02 x
e3 — ring_03 x	e4 — ring_04 x
e5 — ring_05 x	e6 — ring_06 x
e7 — ring_07 x	e8 — ring_08 x
e9 — ring_09 x	e10 — ring_10 x
e11 — ring_11 x	e12 — ring_12 x
e13 — ring_13 x	e14 — ring_14 x
e15 — ring_15 x	e16 — ring_16 x
e17 — ring_17 x	e18 — ring_18 x

funnel_ladder: ring_01 = 100 V → ring_18 = 75 V (18 taps, step -1.471 V) · gradient unavailable: this model does not expose per-electrode masks (native builders do not set `e1_masks`)

new group name:
 kind



Voltages tab, DC ladder groups: feed a chain at both ends and members are interpolated by their explicit number -- order is never parsed from names.

funnel_ladder — members

e1 — ring_01 x	e2 — ring_02 x
e3 — ring_03 x	e4 — ring_04 x
e5 — ring_05 x	e6 — ring_06 x
e7 — ring_07 x	e8 — ring_08 x
e9 — ring_09 x	e10 — ring_10 x
e11 — ring_11 x	e12 — ring_12 x
e13 — ring_13 x	e14 — ring_14 x
e15 — ring_15 x	e16 — ring_16 x
e17 — ring_17 x	e18 — ring_18 x

funnel_ladder: ring_01 = 100 V → ring_18 = 75 V (18 taps, step -1.471 V) · gradient unavailable: this model does not expose per-electrode masks (native builders do not set `e1_masks`)

new group name:
 kind

Numbers members by centroid-z from the SOLVE masks

Per-electrode — DC always applies; assign an RF group or (none). A DC-group member's DC is DERIVED from the ladder (read-only) — set its number instead.

e1 — ring_01

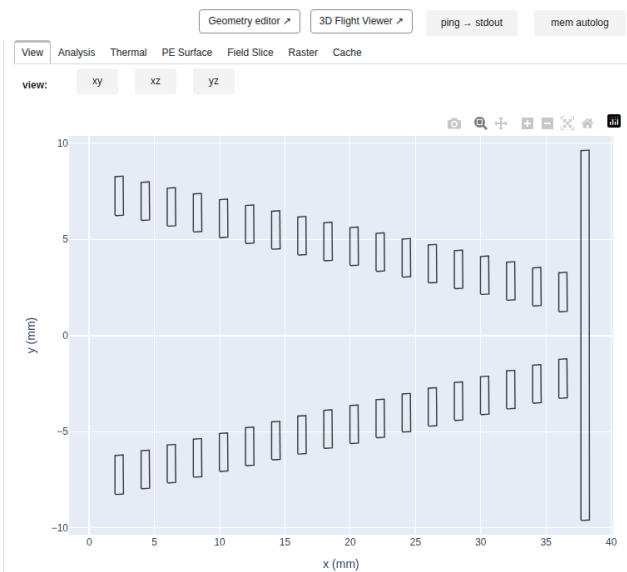
DC (V) — from ladder

DC group
 number

e2 — ring_02

DC (V) — from ladder

DC group
 number



Voltages tab, ladder numbering: auto-number by z orders members by centroid-z straight from the SOLVE masks -- it needs a solved field and refuses rather than guessing.

Load drive template (voltage groups only, from another json):**

Drive groups** (define waveform + amplitude/frequency/phase once; assign electrodes below):

- **RFA waveform (Select)** — Drive waveform for this group. 'table' is the general sampled shape (arbitrary); analytic shapes are sin/square (triangle/sawtooth extensible).
- **RFA amplitude (V) (FloatInput)** — Drive amplitude, 0-to-peak volts (a value of 100 here means 200 V peak-to-peak).
- **RFA frequency (Hz) (FloatInput)** — Drive frequency for this group, in Hz. For a stepped traveling wave this is the per-group rate.
- **RFA phase (deg) (FloatInput)** — Phase offset of this group's waveform. A traveling wave is groups stepped in phase; two confinement rails run 180 deg apart.
- **RFA duty (FloatInput)** — Square waves only: fraction of the period HIGH (0.5 = 11110000 across 8 stepped phases; 0.25 = 11000000).
- **RFA pulse delay τ (μ s) (FloatInput)** — Step-table drives only: the waveform holds 0 until τ , then 1 ($\times$ amplitude). This is Wiley-McLaren delayed extraction when it drives the pusher/extraction plates. NOTE: τ is co-tuned with the mirror detune — changing τ alone moves the velocity focus off the detector. Non-step tables are read-only here.
- **RFA — members (MultiChoice)** — Electrodes driven by this group. Same as the per-electrode 'drive groups' box; an electrode may be in several drive groups and a DC group.
- **RFB waveform (Select)** — Drive waveform for this group. 'table' is the general sampled shape (arbitrary); analytic shapes are sin/square (triangle/sawtooth extensible).
- **RFB amplitude (V) (FloatInput)** — Drive amplitude, 0-to-peak volts (a value of 100 here means 200 V peak-to-peak).
- **RFB frequency (Hz) (FloatInput)** — Drive frequency for this group, in Hz. For a stepped traveling wave this is the per-group rate.
- **RFB phase (deg) (FloatInput)** — Phase offset of this group's waveform. A traveling wave is groups stepped in phase; two confinement rails run 180 deg apart.
- **RFB duty (FloatInput)** — Square waves only: fraction of the period HIGH (0.5 = 11110000 across 8 stepped phases; 0.25 = 11000000).
- **RFB pulse delay τ (μ s) (FloatInput)** — Step-table drives only: the waveform holds 0 until τ , then 1 ($\times$ amplitude). This is Wiley-McLaren delayed extraction when it drives the pusher/extraction plates. NOTE: τ is co-tuned with the mirror detune — changing τ alone moves the velocity focus off the detector. Non-step tables are read-only here.
- **RFB — members (MultiChoice)** — Electrodes driven by this group. Same as the per-electrode 'drive groups' box; an electrode may be in several drive groups and a DC group.
- new group name (TextInput)
- waveform (Select)
- + add drive group (Button)
- remove group(s) (MultiChoice)
- - remove selected (Button)

traveling-wave builder

- **TW ladder electrodes (in order) (MultiChoice)** — Ordered electrodes the wave travels along. Order is the wave direction; phase steps one electrode per segment. Assignment is cyclic (electrode $j \rightarrow$ phase $j \bmod N$), so any number of segments needs only N groups.
- **phases (N) (IntInput)** — Number of distinct phases (4 and 8 are common SLIM).
- **waveform (Select)** — square = classic stepped SLIM TW; sin = sinusoidal TW.
- TW freq (kHz) (FloatInput)
- amp (V) (FloatInput)

- **offset (V) (FloatInput)** — DC offset of the drive. Unipolar 0..V SLIM drive = amp V/2, offset V/2 (e.g. amp 25, offset 25 -> 0..50 V).
- prefix (TextInput)
-  build traveling wave (Button)
- **↔ retune all TW phases (Button)** — Apply the amp / freq / waveform / offset above to EVERY phase group of the existing ladder at once, keeping each group's phase. Use after 'build traveling wave' to change the drive with one edit instead of N.

DC groups — feed a chain at both ends; members are interpolated by their *number*:**

- **funnel_ladder — DC in (V) (FloatInput)** — Applied to the member with the LOWEST number.
- **funnel_ladder — DC out (V) (FloatInput)** — Applied to the member with the HIGHEST number. A RE-WEIGHT, not a re-solve: cache hit.
- **funnel_ladder — members (MultiChoice)** — Electrodes in this DC group. Editing here is the same as the per-electrode 'DC group' dropdown — an electrode may also be in a drive group at the same time.
- new group name (TextInput)
- **kind (Select)** — ladder = two-end interpolated divider (set DC in / DC out and a number per member). uniform = one voltage on every member (no numbering needed).
- + add DC group (Button)
- remove DC group(s) (MultiChoice)
- - remove selected (Button)
- **auto-number by z (Button)** — Number the members of each group in ascending centroid-z, straight from the SOLVE masks. Needs a solved field; refuses rather than guessing.

e1 — ring_01 (and identically for every electrode)

- **DC (V) — from ladder (FloatInput)** — Static (DC) potential on this electrode. When the electrode belongs to a DC group this is DERIVED from the group's in/out and its number.
- **drive groups (MultiChoice)** — Assign this electrode to zero or MORE drive groups (multi-select). An electrode can carry a confinement RF and a traveling-wave AC at once, plus its DC.
- **DC group (Select)** — Ladder membership. DC and RF groups are independent — an electrode can be in both.
- **number (IntInput)** — Position in the ladder. EXPLICIT: order is never parsed from the name (E10 would sort before E9). DC in lands on the lowest, DC out on the highest.

The per-electrode block repeats identically for each electrode in the stack: its DC value (derived from the ladder when the electrode is a member), its drive-group memberships, its DC group and its explicit ladder number. Only the first is shown above; restating nineteen copies would document the layout, not the controls.

Physics

Fly

Stop

Clear ions

Recompute field

Reset app

Ion Source

Voltages

Physics

Display

Impact Analysis

Geometry Import

Status

Multi FA

Config

Gas

Integration

Bounds

☒ collisions enabled

collision model ⓘ

hard-sphere (HS)

HS = discrete elastic; SDS = mobility drift + diffusion

buffer gas

N2

gas temperature (K) ⓘ

273

P (Torr)

2.5

Buffer-gas pressure

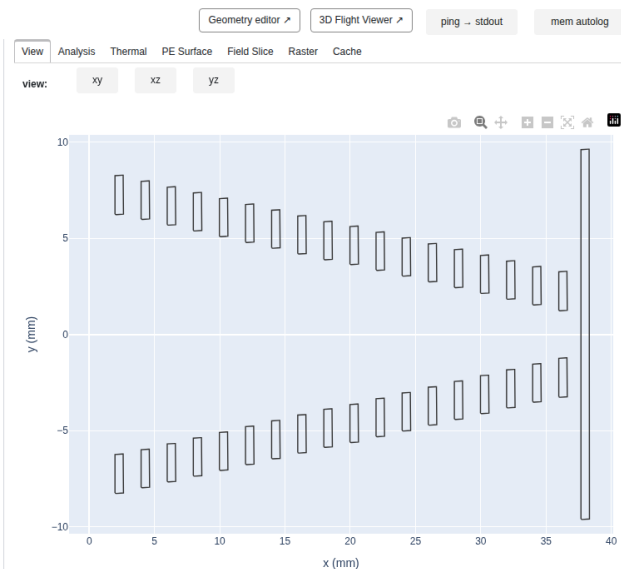
dt guidance — fastest RF period 1176.5 ns (resolve: $dt \leq 23.53$ ns); mean free path 0.005 mm at $v=1.76$ mm/ μ s → collision time 2.8 ns (Poisson sampling: $dt \leq 0.28$ ns). Recommended $dt \leq 0.28$ ns; current $dt = 2$ ns ⚠

HS cross-section σ (m²) ⓘ

2.27e-18

SDS gas diameter (nm) ⓘ

0.366



Physics tab, Gas sub-tab: the collision model (HS hard-sphere or SDS diffusion), the buffer gas, its pressure and temperature. The dt guidance line underneath is live -- it derives the resolving and collision-sampling limits from the current gas and drive, and warns when the declared step is too coarse for either.

Fly

Stop

Clear ions

Recompute field

Reset app

Ion Source

Voltages

Physics

Display

Impact Analysis

Geometry Import

Status

Multi FA

Config

Gas

Integration

Bounds

time step Δt (ns) ⓘ

2

Must resolve the fastest RF (~1/20 of the period)

max flight time (μ s) ⓘ

3000

Ions still airborne here stop with fate 'time out'

record every N steps ⓘ

10

max recorded points per ion ⓘ

100000

extra recorded quantities ⓘ

speed | ×

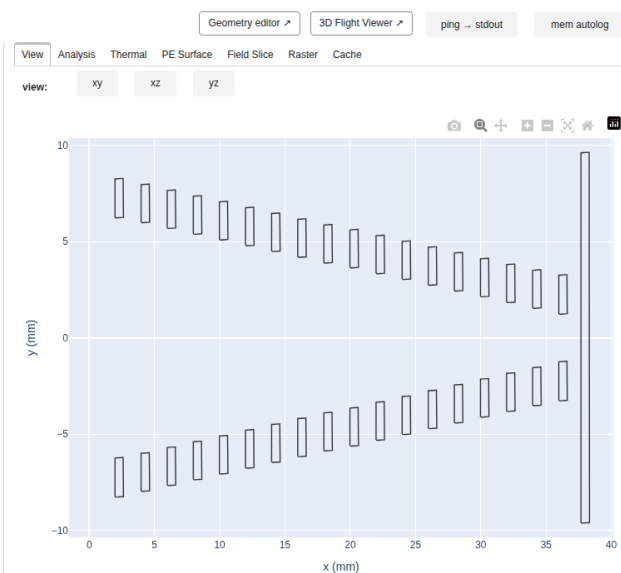
ke_ev | ×

ϕ_{field} | ×

long-run storage — trajectories can be large; turn storage off to just fly ions and keep only their fates. (Clear stored runs is on Config > Runs > maintenance.)

☒ store trajectories (off = fly for fates only, low memory)

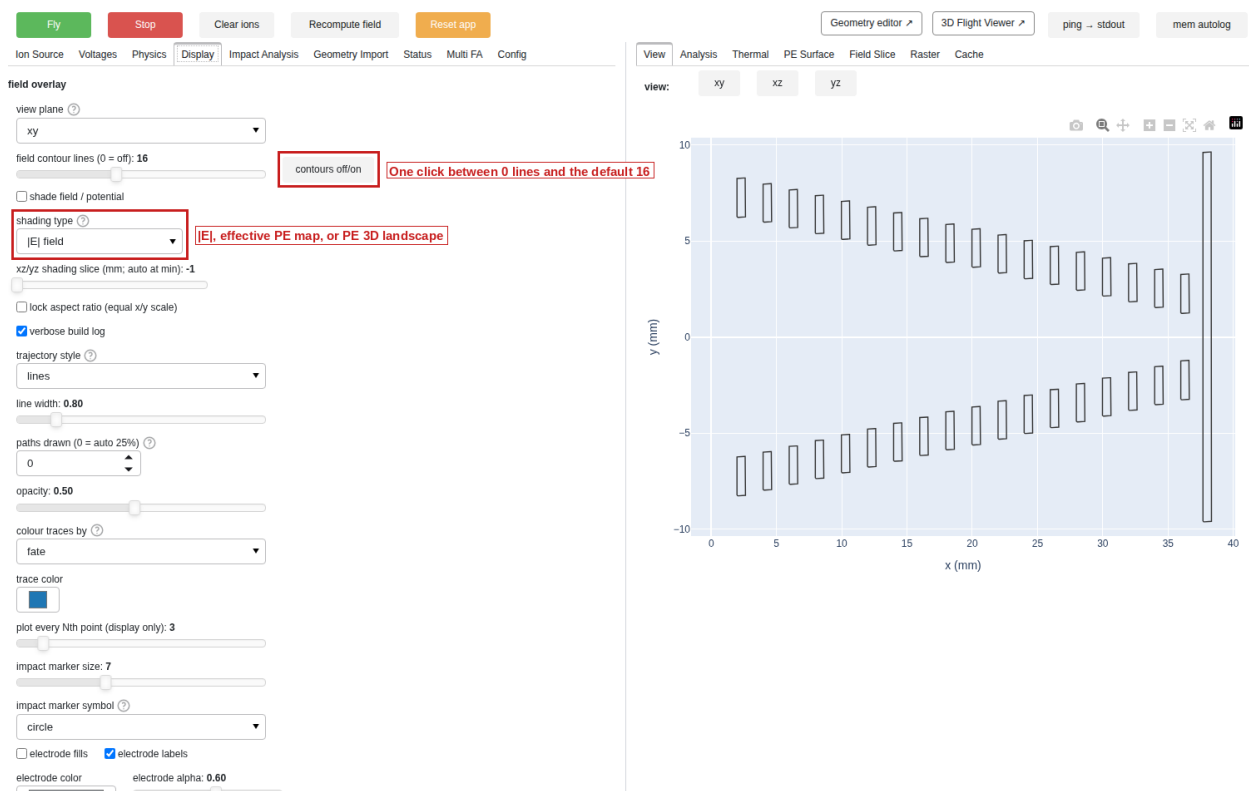
☒ clear previous stored runs on each Fly



Physics tab, Integration sub-tab: the integration time step and the flight-time ceiling. This is a separate capture because it is a separate sub-tab -- a callout can only box a control the reader can actually see.

- collisions enabled (Checkbox)
- **collision model (Select)** — HS = discrete elastic hard-sphere collisions. SDS = the Statistical Diffusion model (mobility drift + diffusion), matched to tabulated ion mobilities.
- buffer gas (Select)
- **gas temperature (K) (FloatInput)** — Buffer-gas temperature; sets the thermal velocity ions relax toward.
- P (Torr) (FloatInput)
- **HS cross-section σ (m²) (LiteralInput)** — Hard-sphere ion–neutral collision cross-section. Used only by the HS model; larger σ = more frequent collisions. Typical N₂ value $\approx 2.3\text{e-}18$ m².
- **SDS gas diameter (nm) (FloatInput)** — Neutral gas molecule diameter, used only by the SDS (diffusion) model to set the mean free path. Air ≈ 0.366 nm.
- **time step Δt (ns) (FloatInput)** — Integration time step. Smaller = more accurate but slower; it must resolve the fastest RF (rule of thumb $\approx 1/20$ of the RF period, so ≈ 0.06 μs at 800 kHz).
- **max flight time (μs) (FloatInput)** — Longest an ion is flown. An ion still airborne at this time stops with fate 'time out'.
- **record every N steps (IntInput)** — Trajectory sampling interval: one point is stored every N integration steps. Larger = fewer stored points (smaller files, faster plots); it does NOT change the physics, only how densely the path is saved.
- **max recorded points per ion (IntInput)** — Trajectory buffer cap. When a flight needs more points than this the RECORDING stops but the ion keeps flying — the app warns before the fly and names the true stopping point afterwards. Raise this (or 'record every N') to capture a whole long flight.
- **extra recorded quantities (MultiChoice)** — Additional per-point values to store along each trajectory (e.g. field magnitude, collision count) for coloring or export.
- store trajectories (off = fly for fates only, low memory) (Checkbox)
- clear previous stored runs on each Fly (Checkbox)
- x min on (Checkbox)
- x min (mm) (FloatInput)
- x max on (Checkbox)
- x max (mm) (FloatInput)
- y min on (Checkbox)
- y min (mm) (FloatInput)
- y max on (Checkbox)
- y max (mm) (FloatInput)
- z min on (Checkbox)
- z min (mm) (FloatInput)
- z max on (Checkbox)
- z max (mm) (FloatInput)

Display

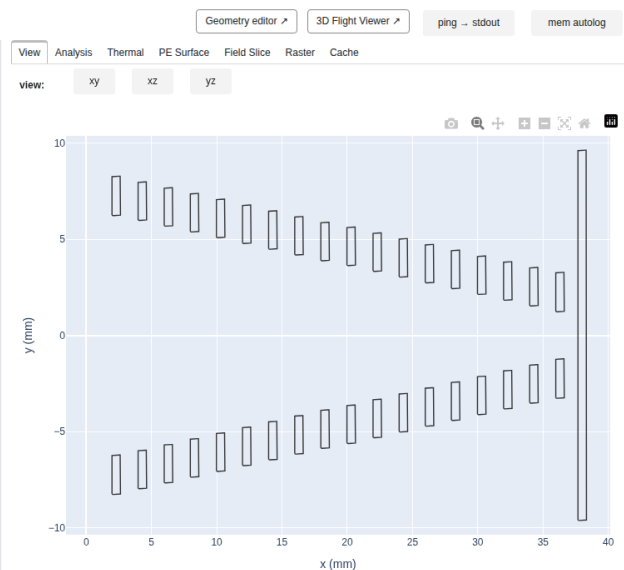
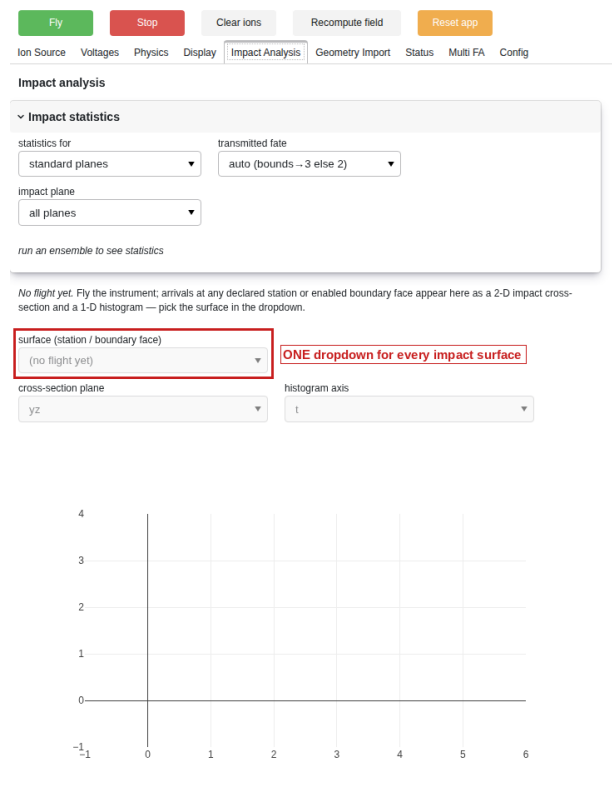


Display tab: what the view pane draws and how. The field overlay block at the top drives the shading and its contour lines; the contour slider is the single value authority and the off/on button flips it between 0 and the default of 16.

- **view plane (Select)** — Which 2-D plane to project the trajectories and field onto for display.
- equipotential lines (0=off) (IntSlider)
- shade field / potential (Checkbox)
- **shading type (Select)** — What the shading shows: instantaneous field magnitude $|E|$, the effective RF pseudopotential (PE) as a 2-D map, or the PE as a 3-D landscape.
- xz/yz shading slice (mm; auto at min) (FloatSlider)
- lock aspect ratio (equal x/y scale) (Checkbox)
- verbose build log (Checkbox)
- **trajectory style (Select)** — Draw each ion path as connected lines, individual recorded points, or both.
- line width (FloatSlider)
- opacity (FloatSlider)
- **color traces by (Select)** — Color each trajectory by its outcome (fate), by m/z (one color per mass, with a legend), a single color, or a recorded channel.
- trace color (ColorPicker)
- plot every Nth point (display only) (IntSlider)
- impact marker size (FloatSlider)
- **impact marker symbol (Select)** — Marker shape drawn where an ion terminates on metal or a bounding plane.
- electrode fills (Checkbox)
- electrode labels (Checkbox)
- electrode color (ColorPicker)

- electrode alpha (FloatSlider)
- Show field (no ions) (Button)

Impact Analysis



Impact Analysis tab before any flight: the surface dropdown enables once arrivals exist. Stations and enabled boundary faces share one stream and one naming authority (enabled_planes), so the dropdown and the stats table can never disagree.

Impact analysis

- statistics for (Select)
- transmitted fate (Select)
- impact plane (Select)
- surface (station / boundary face) (Select)
- cross-section plane (Select)
- histogram axis (Select)

Geometry Import

Fly

Stop

Clear ions

Recompute field

Reset app

Ion Source

Voltages

Physics

Display

Impact Analysis

Geometry Import

Status

Multi FA

Config

STL Upload

Drag & Drop your files or Browse

electrode

dc [V]

rf_group

mesh faces

pitch mm/gu (you set)

0.5

Preview voxelization (coarse)

Commit geometry

☐ solve full 3-D (slower; otherwise 2-D mid-z slice)

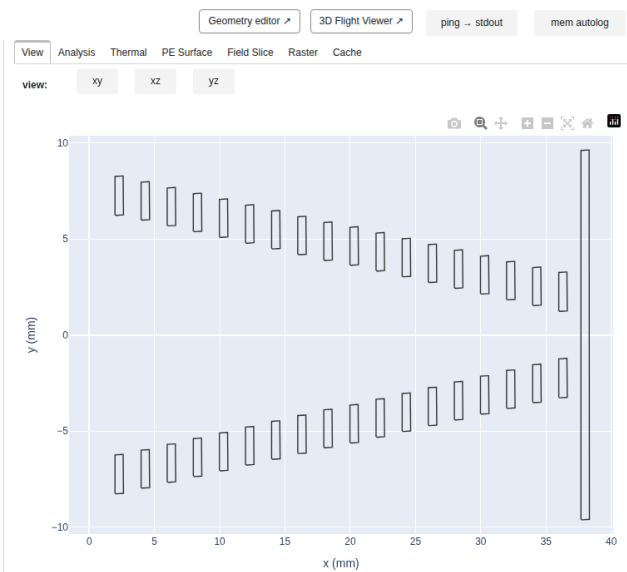
drop one STL per electrode; set a pitch to see the solve cost

Clear loaded STLs + cache

4

3

2



Geometry Import tab: bring electrodes in from JSON or STL.

- pitch mm/gu (you set) (FloatInput)
-  Preview voxelization (coarse) (Button)
- Commit geometry (Button)
- solve full 3-D (slower; otherwise 2-D mid-z slice) (Checkbox)
- Clear loaded STLs + cache (Button)

Status

FlyStopClear ionsRecompute fieldReset app✓ 24/24 ions in 2 s

Ion Source Voltages Physics Display Impact Analysis Geometry ImportStatusMulti FAConfig

current

histogram of x (20 bins).

workers

worker threads 1

1 thread(s) · 0 in flight · 0 queued · 24/24 done · 129.2 ions/s

history (this session, newest first, last 100)

21:01:29

histogram of x (20 bins).

21:01:21

histogram of t (20 bins).

21:01:08

plotted x vs t.

21:00:58

done: 24 ions in 0.2 s (12 timeout, 9 impact / exit, 3 bounding plane) · median tof 15.46 μs · stored as 'Quadrupole (2-D analytic rods)' [21:00:58] (reloadable) · /flight #1

21:00:58

24/24 ions — 129/s

21:00:57

flying 24 ions... (a first fly in a fresh session pauses ~30-60 s compiling the tracer before ion 1 — Stop takes effect after that)

21:00:56

loading cached field... 0s elapsed (one-time for this geometry; voltage & view changes after this are instant)

Status tab after flying Quadrupole (2D, analytic): the run log names the solve, the fly, and the fates.

FlyStopClear ionsRecompute fieldReset app

Ion Source Voltages Physics Display Impact Analysis Geometry ImportStatusMulti FAConfig

current

idle

workers

worker threads 1

no flight this session

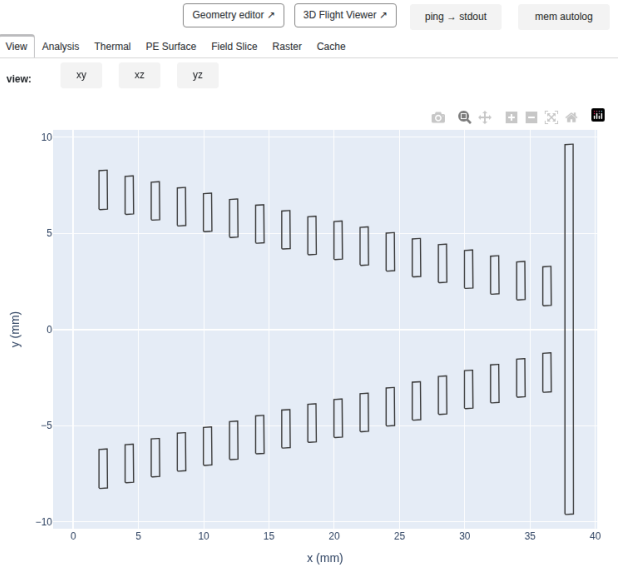
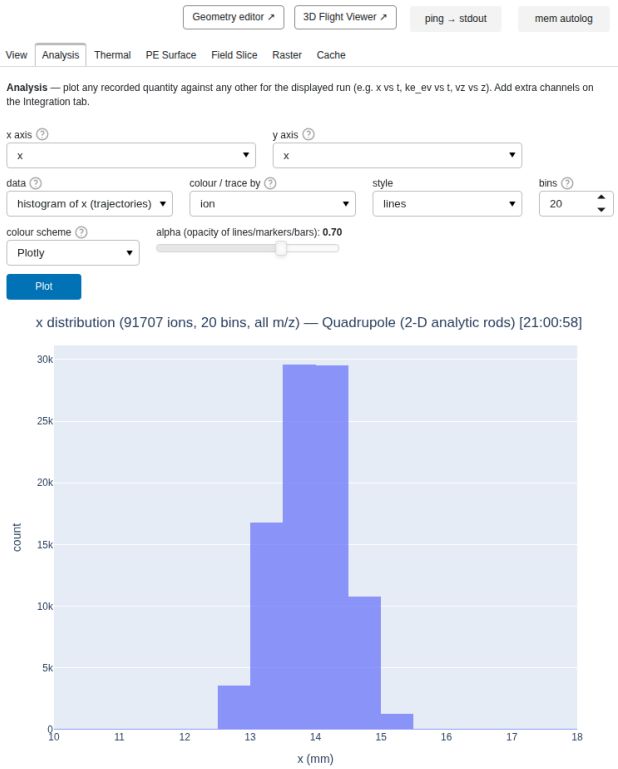
history (this session, newest first, last 100)

no messages yet this session

Status tab: the run log.

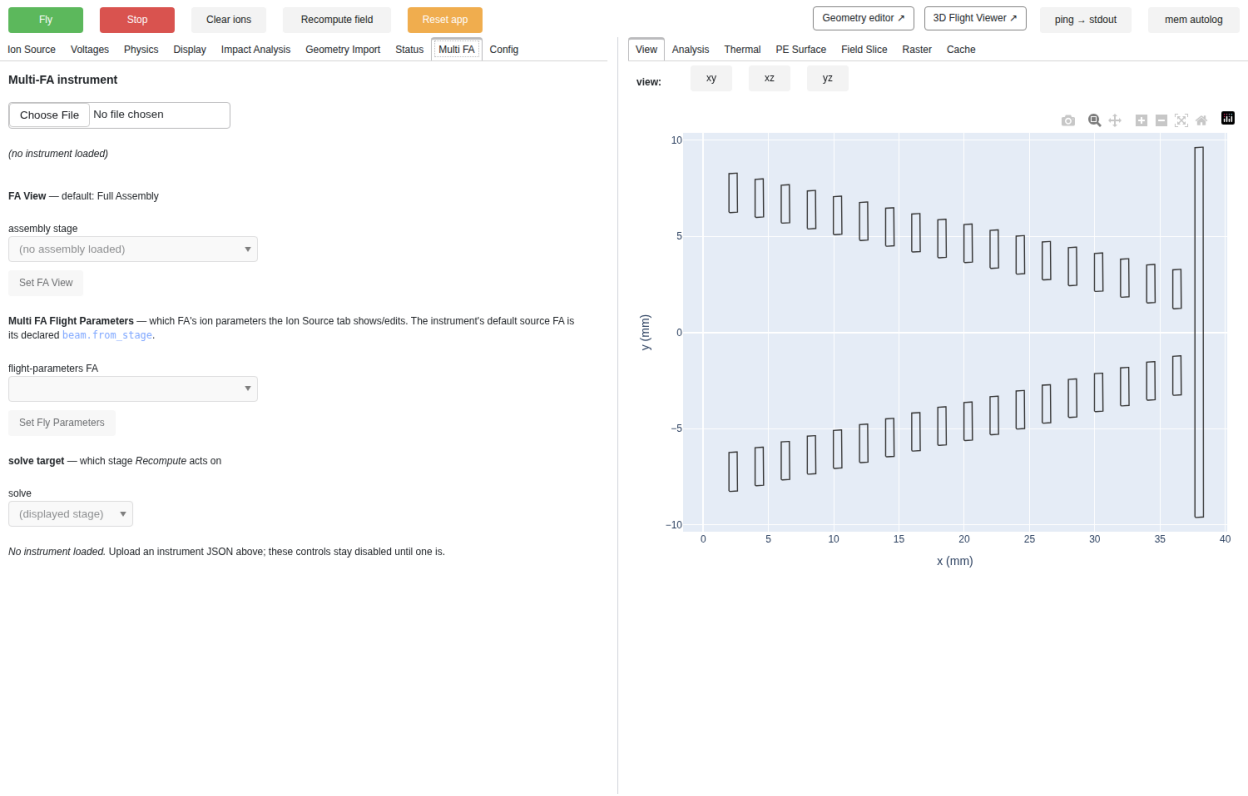
current

idle



history *(this session, newest first, last 100)

Multi FA



Multi FA tab: staged assemblies -- each stage solves in its own grid and hands the ion across a declared seam.

Multi-FA instrument

FA View** — default: Full Assembly

- assembly stage (Select)
- Set FA View (Button)
- flight-parameters FA (Select)
- Set Fly Parameters (Button)

solve target** — which stage *Recompute* acts on

- solve (Select)

Config

- **save as (filename) (TextInput)** — Filename for the downloaded configuration. Include the .json extension (added if omitted).
- Download spec (FileDownload)

stored runs (reloadable)

- stored runs (Select)
- Reload run into view (Button)
- Export displayed run (FileDownload)
- Download run (NPZ) (FileDownload)
- **Export view + run (HTML) (FileDownload)** — Self-contained HTML of the CURRENT view (field / PE surface + ion trajectories) with the full run metadata (geometry, voltages, gas, integration, fates) embedded.
- **Save solved field (Button)** — Save the current geometry's solved bases as one named .npz in the fields folder (self-validating; safe to rename or send).
- **Save trajectories (reloadable here) (Button)** — Save the ACTIVE run's ion paths to this app's store as a named .traj.npz; reload it with 'Load as run' or open it in notebook 05. Stays server-side (see the save path in the Status tab). For a file to take elsewhere, use 'Download run (NPZ)' above.
- saved fields (Select)
- ↻ (Button) — Rescan the fields folder.
- **Load field (Button)** — Validate the selected field against the LOADED geometry and reinstate it into the cache. Refuses on any mismatch — a wrong-geometry field is silently wrong physics.
- **Open saved field (Button)** — One-file session open (PI 2026-08-06): reconstruct the geometry + operating point from the file's embedded spec, load it into the editor as if you had pasted the JSON, then validate and reinstate the bases. Refuses pre-bootstrap saves (no embedded spec) with the reason.
- **label (TextInput)** — Local display label for the selected field (stored in the registry; never rewrites the npz).
- **Relabel (Button)** — Apply the label to the selected field (local registry only).
- saved trajectories (Select)
- **Load as run (Button)** — Load the selected trajectory file as a stored run (overlays even across geometries, with a warning).
- Clear stored runs now (Button)
- Clear field cache (Button)

View (plot pane)

view:

- xy (Button)
- xz (Button)
- yz (Button)

Analysis (plot pane)

- **x axis (Select)** — Recorded channel for the horizontal axis.
- **y axis (Select)** — Recorded channel for the vertical axis.
- **data (Select)** — Plot every recorded point, one point per ion at its final state, or a histogram of the x-axis quantity over per-ion endpoints (y axis is ignored).
- **color / trace by (Select)** — ion: a separate color per ion (cycling palette). all together: one series. fate: color by outcome. m/z : one color per m/z value — each ion is colored by its OWN flown m/z (stable palette, sorted numerically, shown in the legend); works in all three plot modes.

- style (Select)
- **bins (IntInput)** — Histogram bin count. A good rule of thumb is about $\sqrt{\text{number of ions}}$; the default 20 suits ~50-400 ions. Only used in histogram mode.
- ***m/z* filter (MultiChoice)** — Empty = ALL *m/z* flown (default). Pick one or more values to plot only those ions. Options come from the displayed run's FLOWN ions (each ion's own recorded *m/z*), not from the editor — so a reloaded older run offers what it actually flew.
- **color scheme (Select)** — Palette for line/marker colors — applies to per-ion cycling, *m/z* coloring (stable assignment over sorted *m/z*), and the all-together series. Fate colors stay fixed: they are semantic (impact/alive/timeout), not aesthetic.
- alpha (opacity of lines/markers/bars) (FloatSlider)
- Plot (Button)

Thermal (plot pane)

- T_slope tail-fit percentile (FloatSlider)
- **bins (IntInput)** — Histogram bins for the velocity and energy distributions.
- Assess temperature (Button)
- ions (IntInput)
- cell size (mm) (FloatInput)
- hot threshold (eV; blank = 10 kT) (FloatInput)
- Map hot spots (Button)

PE Surface (plot pane)

- ion *m/z* (Da) (FloatInput)
- component (Select)
- plane (Select)
- slice position (mm; auto if at min) (FloatSlider)
- slice (mm, exact) (FloatInput)
- electrodes on the PE scale (Select)
- trust radius (cells from metal) (IntSlider)
- drape current trajectories (Checkbox)
- height (z aspect) (FloatSlider)
- PE transform (optional) (Select)
- PE transform factor / asinh knee (eV) (FloatInput)
- resolution (pts/axis, ↓ = faster) (IntSlider)
- Compute surface (Button)

Field Slice (plot pane)

- quantity (Select)
- ion *m/z* (Da) (FloatInput)
- plane (Select)
- slice position (mm; auto if at min) (FloatSlider)
- slice (mm, exact) (FloatInput)

heatmap

- heatmap (Checkbox)
- heatmap colors (Select)
- log scale (Checkbox)
- range min (blank=auto) (FloatInput)
- range max (blank=auto) (FloatInput)

contours

- contours (Checkbox)
- # contours (IntSlider)
- contour colors (Select)
- resolution (pts/axis, ↓ = faster) (IntSlider)
- lock aspect (true mm) (Checkbox)

view window** (mm; blank = full domain)

- x min (blank=full) (FloatInput)
- x max (FloatInput)
- y min (blank=full) (FloatInput)
- y max (FloatInput)
- reset view (Button)
- Compute slice (Button)
- CSV filename (TextInput)
- ↓ export slice CSV (FileDownload)

Raster (plot pane)

- **Compute 3D View (Button)** — Render the labeled voxel grid the SOLVER consumes (route-agnostic: scene CSG, STL voxelization). Multi-axis: 3-D cloud + exact xy/xz/yz projections.
- show electrodes (Checkbox)
- show legend (Checkbox)

Cache (plot pane)

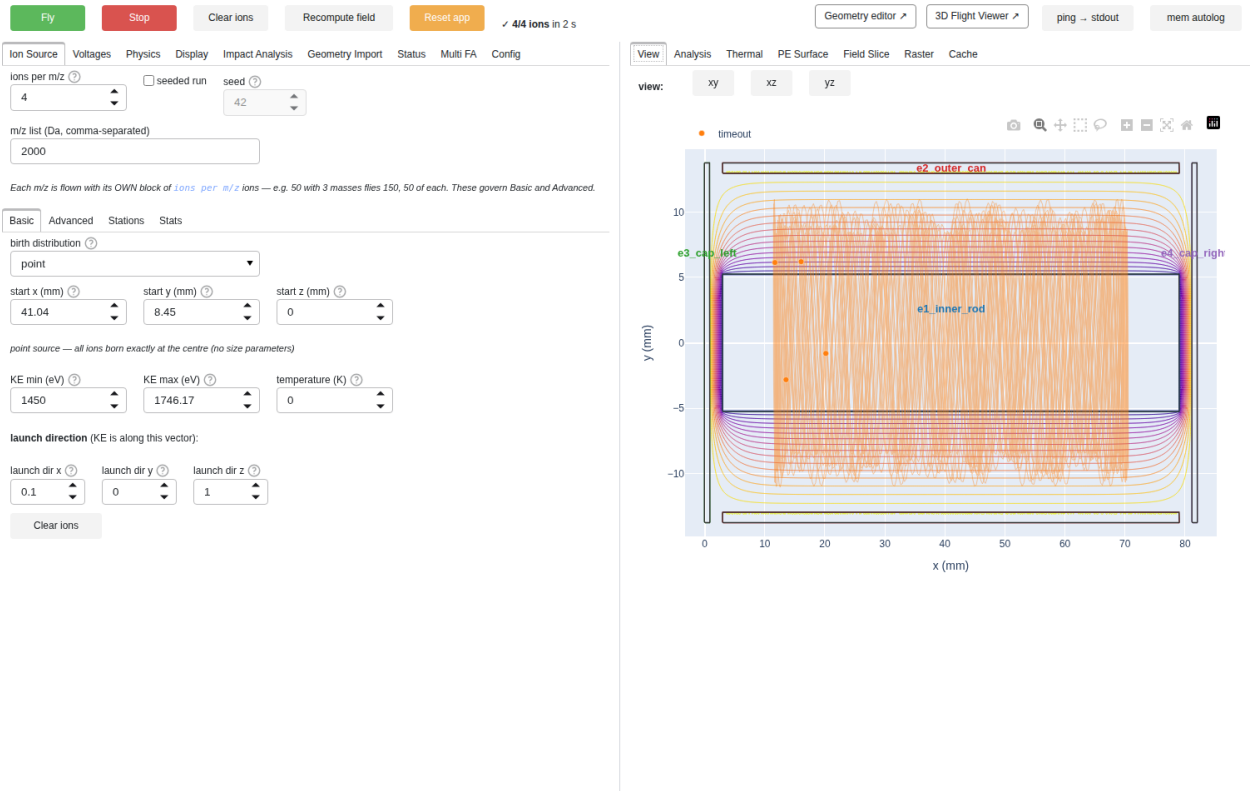
Cache location: `/tmp/manual\_harvest` **1 entries, 0.0 MB total on disk**

- ↻ refresh (Button)
- 🗑 clear ALL field cache (Button)
- entry to remove (Select)
- ✕ remove selected (Button)
- export to (TextInput)
- ↓ export selected (field + spec json) (Button)

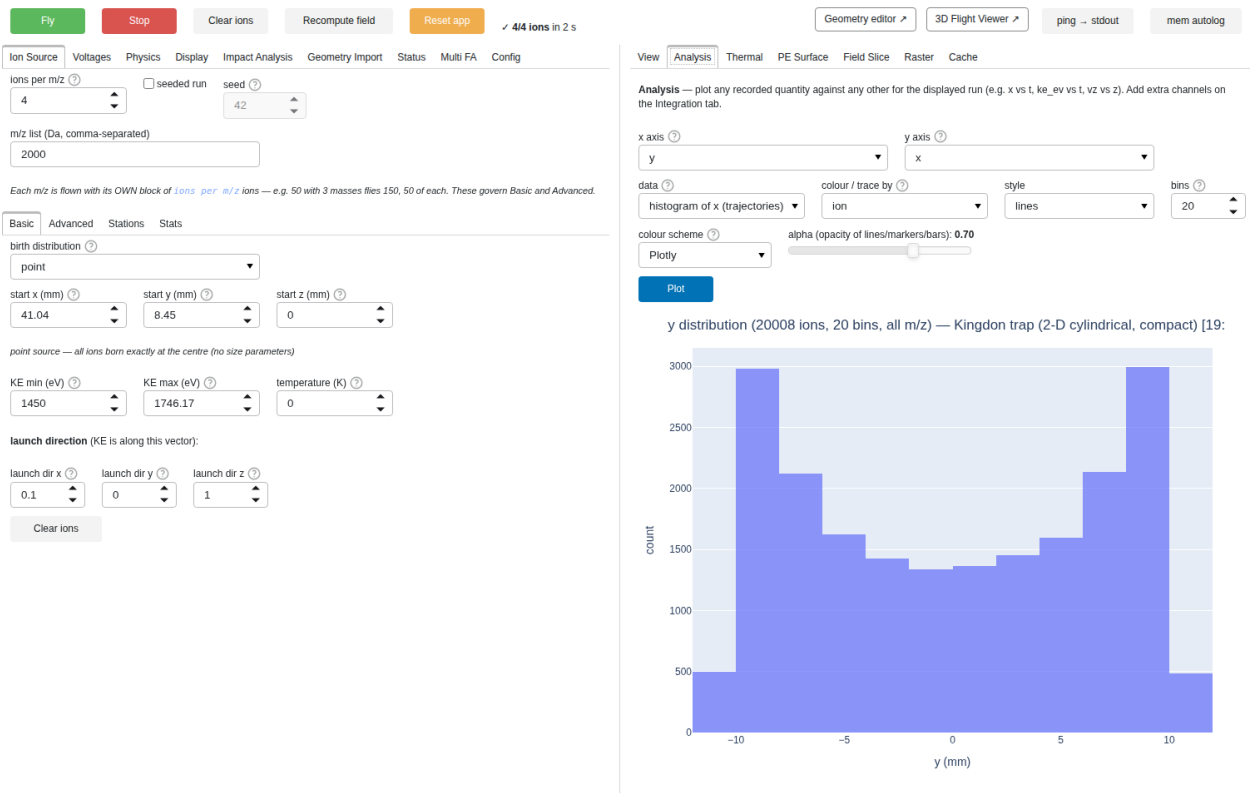
In-memory caches (freed on 'clear ALL' or when the app restarts): - compose gradients: 0 geometry, 0.0 MB**

Worked example captures

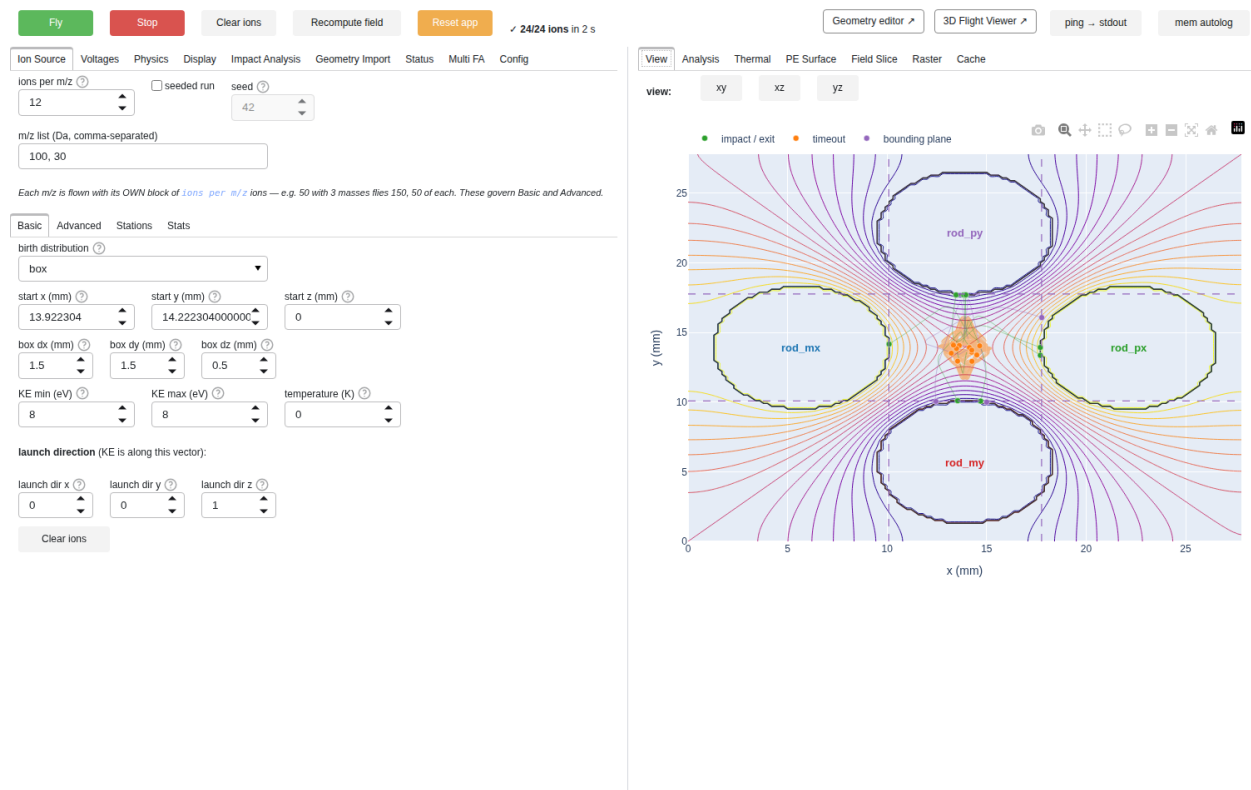
Each capture below is a real flight of a shipped example deck, driven through the dashboard by the screenshot pipeline (docs/capture_ui_screens.py) -- nothing is mocked, and the set regenerates identically after any UI change by re-running the capture sessions.



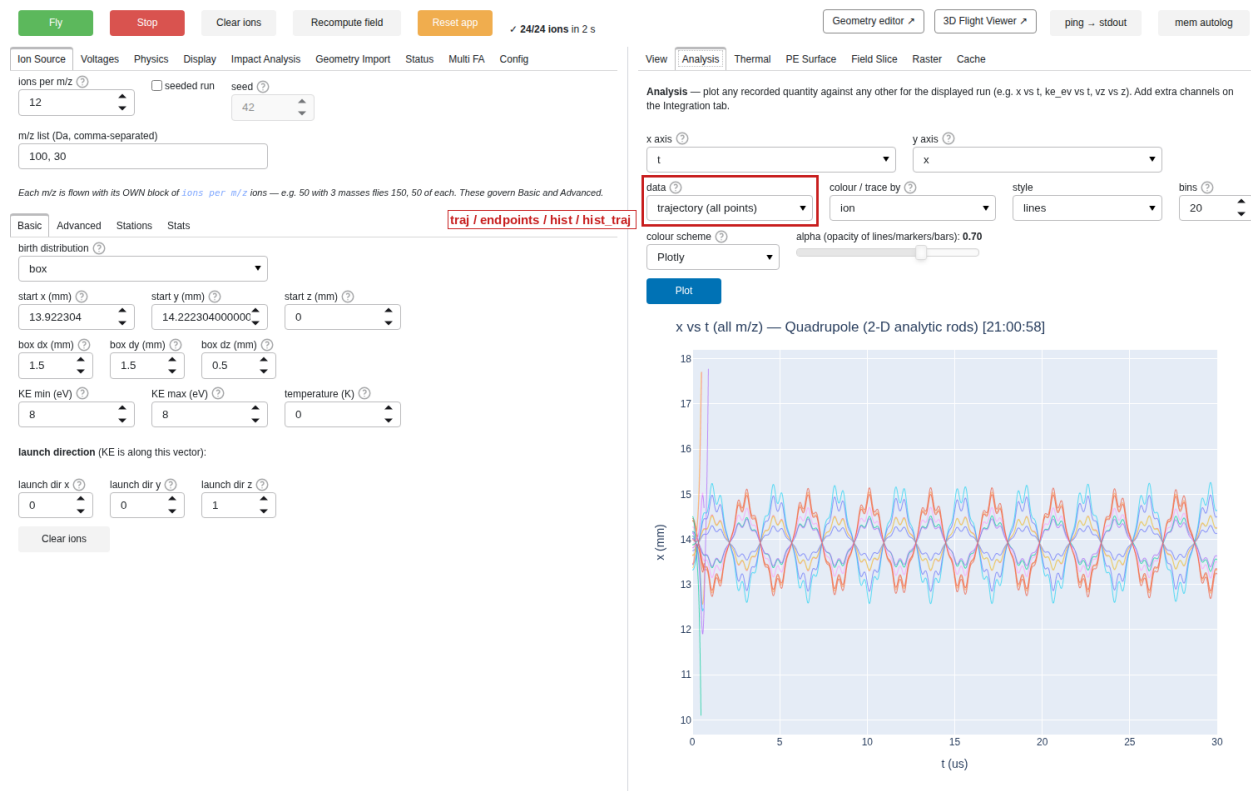
Kingdon Trap (r-z): orbital motion around the central wire (0.65 r / 0.76 z rescale verified).



Kingdon Trap (r-z): hist_traj -- the trajectory-sample histogram of the radius (y in r-z) -- the mode shipped for orbit dwell structure; the pile-up sits at the radial turning points.



Quadrupole (2D, analytic): stable and ejected masses in one fly -- the analytic rod field the toolkit ships. View pane, xy.



Quadrupole (2D, analytic): Analysis tab, trajectory mode (every recorded point of every ion).

Fly

Stop

Clear ions

Recompute field

Reset app

✓ 24/24 ions in 2 s

Ion Source

Voltages

Physics

Display

Impact Analysis

Geometry Import

Status

Multi FA

Config

ions per m/z

12

☐ seeded run

seed

42

m/z list (Da, comma-separated)

100, 30

Each m/z is flown with its OWN block of *ions per m/z* ions — e.g. 50 with 3 masses flies 150, 50 of each. These govern Basic and Advanced.

Basic

Advanced

Stations

Stats

birth distribution

box

start x (mm)

13.922304

start y (mm)

14.222304000000

start z (mm)

0

box dx (mm)

1.5

box dy (mm)

1.5

box dz (mm)

0.5

KE min (eV)

8

KE max (eV)

8

temperature (K)

0

launch direction (KE is along this vector):

launch dir x

0

launch dir y

0

launch dir z

1

Clear ions



Quadrupole (2D, analytic): Analysis tab, per-ion endpoints.

Fly

Stop

Clear ions

Recompute field

Reset app

✓ 24/24 ions in 2 s

Ion Source

Voltages

Physics

Display

Impact Analysis

Geometry Import

Status

Multi FA

Config

ions per m/z

12

☐ seeded run

seed

42

m/z list (Da, comma-separated)

100, 30

Each m/z is flown with its OWN block of *ions per m/z* ions — e.g. 50 with 3 masses flies 150, 50 of each. These govern Basic and Advanced.

Basic

Advanced

Stations

Stats

birth distribution

box

start x (mm)

13.922304

start y (mm)

14.222304000000

start z (mm)

0

box dx (mm)

1.5

box dy (mm)

1.5

box dz (mm)

0.5

KE min (eV)

8

KE max (eV)

8

temperature (K)

0

launch direction (KE is along this vector):

launch dir x

0

launch dir y

0

launch dir z

1

Clear ions



Quadrupole (2D, analytic): Analysis tab, histogram over per-ion endpoints.

Fly Stop Clear ions Recompute field Reset app ✓ 24/24 ions in 2 s

Ion Source Voltages Physics Display Impact Analysis Geometry Import Status Multi FA Config

Ions per m/z: 12 ☐ seeded run seed: 42

m/z list (Da, comma-separated): 100, 30

Each m/z is flown with its OWN block of ions per m/z ions — e.g. 50 with 3 masses flies 150, 50 of each. These govern Basic and Advanced.

Basic Advanced Stations Stats

birth distribution: box

start x (mm): 13.922304 start y (mm): 14.222304000000 start z (mm): 0

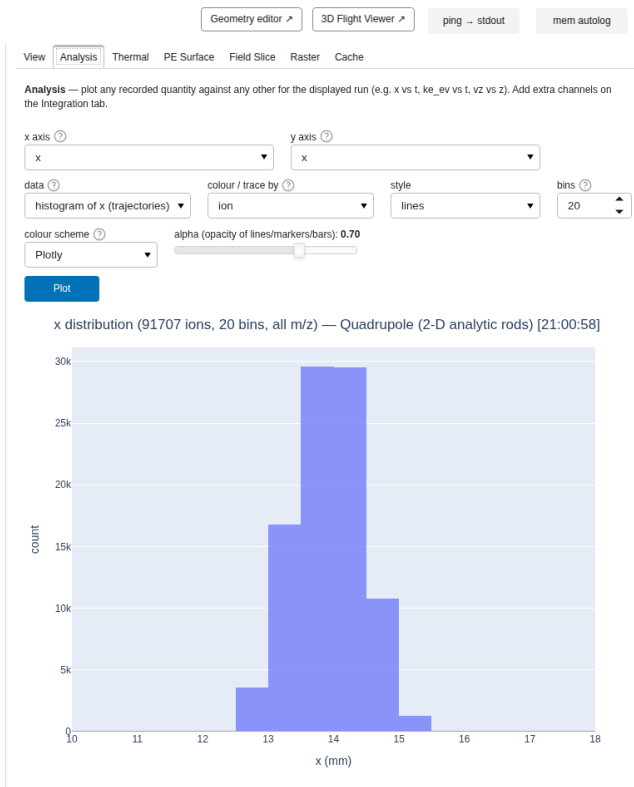
box dx (mm): 1.5 box dy (mm): 1.5 box dz (mm): 0.5

KE min (eV): 8 KE max (eV): 8 temperature (K): 0

launch direction (KE is along this vector):

launch dir x: 0 launch dir y: 0 launch dir z: 1

Clear ions



Quadrupole (2D, analytic): Analysis tab, histogram over EVERY recorded sample (hist_traj) of x -- residence-weighted, so where an ion dwells in the confinement plane is where the counts are.

Fly Stop Clear ions Recompute field Reset app ✓ 24/24 ions in 2 s

Ion Source Voltages Physics Display Impact Analysis Geometry Import Status Multi FA Config

field overlay

view plane: xy

field contour lines (0 = off): 16 contours off/on One click between 0 lines and the default 16

☒ shade field / potential

shading type: |E| field

xz/yz shading slice (mm; auto at min): -1

☐ lock aspect ratio (equal x/y scale)

☒ verbose build log

trajectory style: lines

line width: 0.80

paths drawn (0 = auto 25%): 0

opacity: 0.50

colour traces by: fate

trace color:

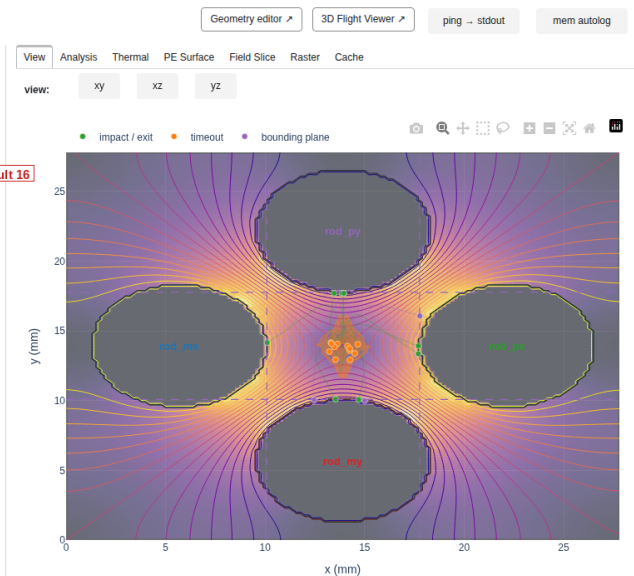
plot every Nth point (display only): 3

impact marker size: 7

impact marker symbol: circle

☐ electrode fills ☒ electrode labels

electrode color: electrode alpha: 0.60



Quadrupole (2D, analytic): the field overlay ON over the same flight -- Display tab, 'shade field / potential' ticked, contour slider at its default of 16. The shading is the instantaneous $|E|$ the ions flew through; the contour lines make the rod quadrupole's saddle structure legible under the trajectories.

[Fly](#)
[Stop](#)
[Clear ions](#)
[Recompute field](#)
[Reset app](#)
✓ 24/24 ions in 2 s

[Ion Source](#)
[Voltages](#)
[Physics](#)
[Display](#)
[Impact Analysis](#)
[Geometry Import](#)
[Status](#)
[Multi FA](#)
[Config](#)

field overlay

view plane [?](#)

xy

field contour lines (0 = off): 0

☐ contours off/on

☒ shade field / potential

shading type [?](#)

|E| field

xz/yz shading slice (mm; auto at min): -1

☐ lock aspect ratio (equal x/y scale)

☒ verbose build log

trajectory style [?](#)

lines

line width: 0.80

paths drawn (0 = auto 25%) [?](#)

0

opacity: 0.50

colour traces by [?](#)

fate

trace color

☐ electrode fills ☒ electrode labels

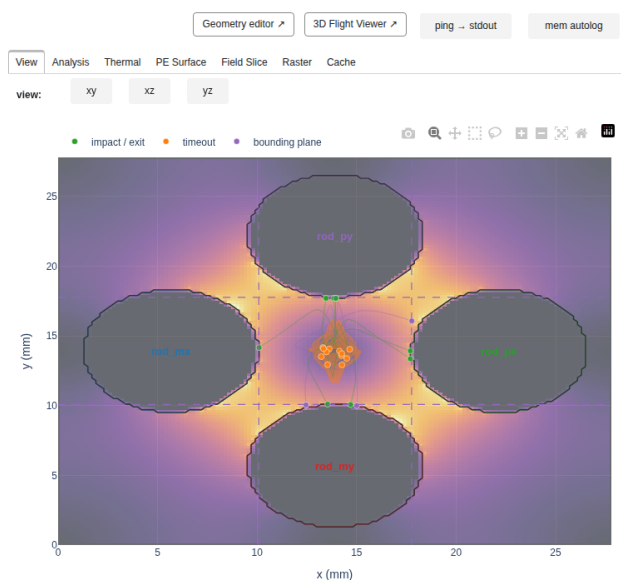
electrode color electrode alpha: 0.60

plot every Nth point (display only): 3

impact marker size: 7

impact marker symbol [?](#)

circle



Quadrupole (2D, analytic): the contours off/on button after one click -- the shading stays, the lines go, and the slider (the single value authority) now reads 0. A second click restores the default 16; dragging the slider sets any count in between.

[Fly](#)
[Stop](#)
[Clear ions](#)
[Recompute field](#)
[Reset app](#)
✓ 48/48 ions in 2 s

[Ion Source](#)
[Voltages](#)
[Physics](#)
[Display](#)
[Impact Analysis](#)
[Geometry Import](#)
[Status](#)
[Multi FA](#)
[Config](#)

ions per m/z [?](#) ☐ seeded run seed [?](#)

24 42

m/z list (Da, comma-separated)

100, 500

Each m/z is flown with its OWN block of ions per m/z ions — e.g. 50 with 3 masses files 150, 50 of each. These govern Basic and Advanced.

Basic Advanced [Stations](#) [Stats](#)

Detector / monitor stations — a plane at `axis = pos` with optional windows over the other axes. `detect` absorbs at first in-window crossing; `record` logs and lets the ion continue.

station [DETECTOR](#) name [DETECTOR](#) kind [detect](#)

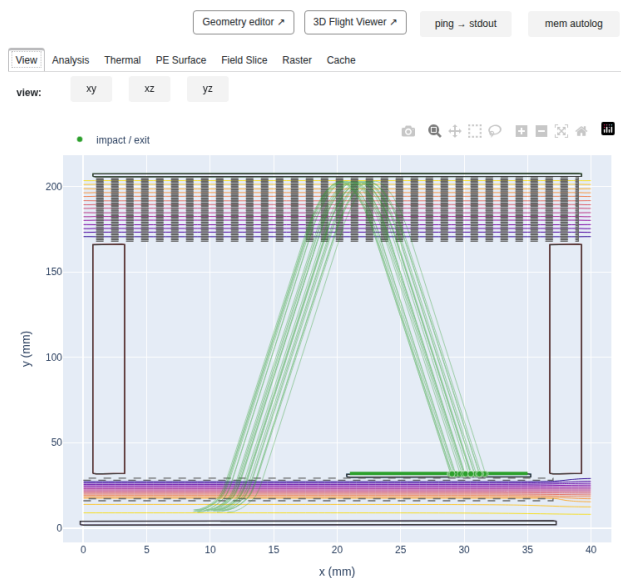
axis [y](#) pos (mm) 32

☒ window x min 21 max 35

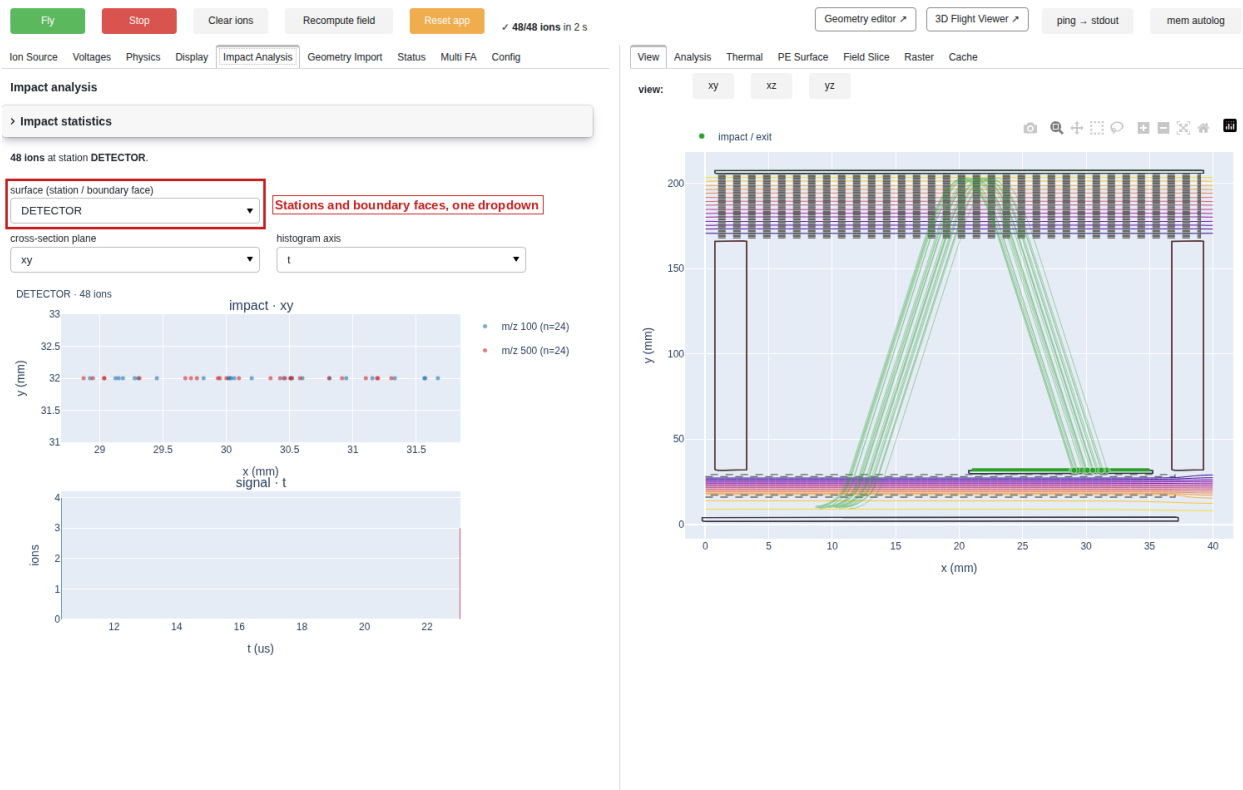
☐ window z min -5 max 5

[Apply station](#) [Delete](#)

station **DETECTOR** applied — in the spec, the JSON box, and the instrument document. Re-fly to see it in Impact Analysis.



Reflectron oa-TOF (2D): orthogonal acceleration, the field-free drift, the ion mirror turnaround and the return to the detector plate, all in one View-pane capture of a real fly (m/z 100 and 500). Electrode labels are turned off here (Display tab checkbox) -- this stack is dense enough that the tags crowd the beam; your own session shows them by default.



Reflectron oa-TOF (2D): Impact Analysis at the DETECTOR station declared on the plate face ($y = 32.0$ mm, windowed to the 21–35 mm plate span). The impact-xy cross-section shows where the packet lands across the plate (this deck is planar, so z is identically zero); the signal-vs-t histogram of the same arrivals is the mass spectrum -- two masses, two peaks.

11.1 The geometry editor (/editor)

What it is. The dashboard's Geometry editor button opens /editor in a new tab, served by the same process. It edits the DECK: a shape is selected by electrode and index, its numbers are typed in millimetres or dragged with the gizmo, and Save writes the JSON back to the path shown at the bottom of the panel. The scene is the metal as the solver will rasterize it, not a sketch of it.

ion_gym geometry editor

document
examples/ion_funnel_rz.json

load a deck by path, or a FOLDER to browse its *.json (resp.-relative or absolute)
The deck being edited (Save writes this file)

Load path / folder

Choose File

No file chosen

electrode

Pick the electrode, then the shape within it

shape
0: rect

☒ snap to pitch
☐ contours (DC potential; solves the 2-D field on demand)
☒ gizmo (drag the selected shape; snaps to pitch)

x_mm
2

y_mm
6.25

width_mm
0.6

height_mm
2

rotation_deg
0

Commits the edited shape into the document

Apply

Undo

Redo

Remove shape

electrode color
Color electrode

Color all

new shape type
rect

Add shape to ring_01

new electrode name
Add electrode

save path
/home/claude/ion_gym_v498_dev/ion_gym_v498_dev/examples/ion_funnel_rz.json

Save

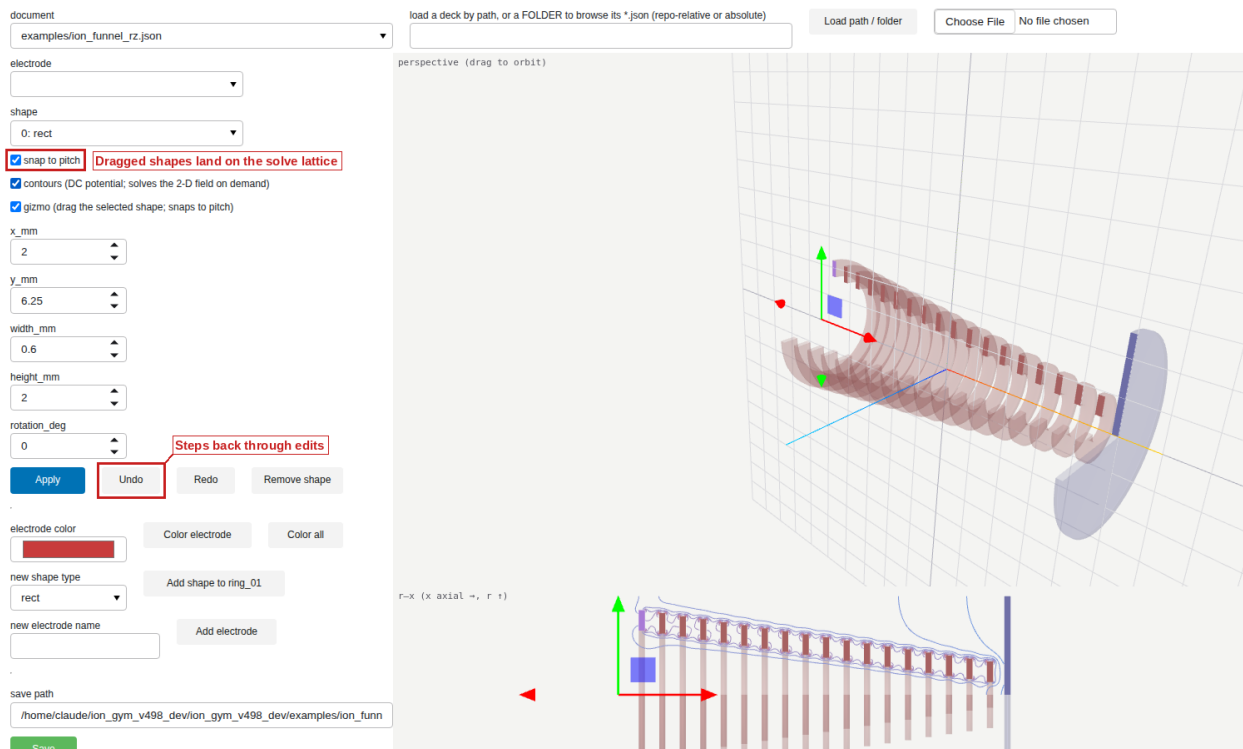
perspective (drag to orbit)

r-x (x axial →, r ↑)

The geometry editor (/editor), opened on the ion funnel deck: the shape being edited is picked by electrode and index on the left, the perspective scene shows the metal as the solver will rasterize it, and the panel beneath it is the r-x section with the axial coordinate horizontal. Edits are applied to the document, not to a copy: Save writes the deck back to the path shown at the bottom.

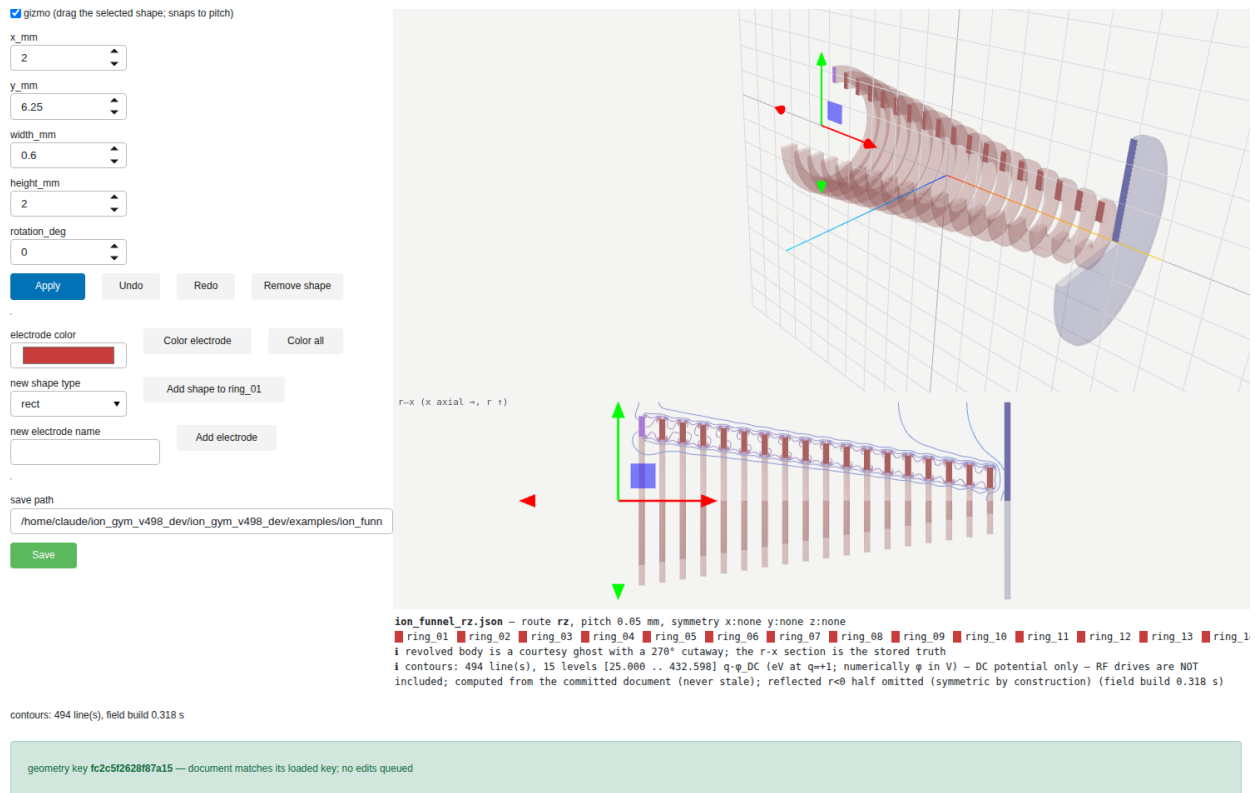
Snap to pitch. With snapping on, a dragged shape lands on the solve lattice rather than between nodes. This matters for the same reason the lattice is declared rather than inferred: metal edges rasterize onto the grid, so an edge dropped half a cell off is a different instrument from the one you meant to draw. Undo and Redo step the document's edit history, and the editor refuses an edit it cannot represent rather than silently rounding it.

ion_gym geometry editor



Snap-to-pitch and the drag gizmo: with 'snap to pitch' on, a dragged shape lands on the solve lattice rather than between nodes -- the editor's way of keeping metal edges where the rasterizer will actually put them. Undo and Redo step the document's edit history.

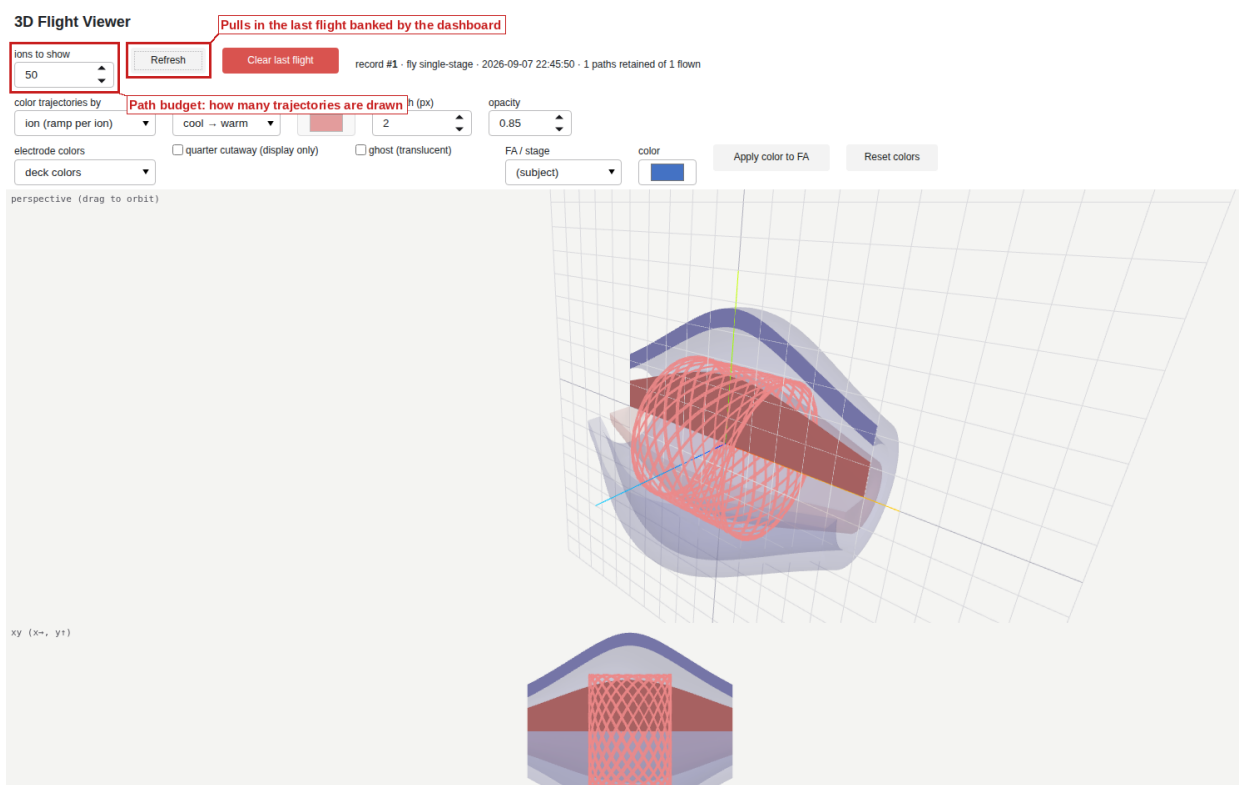
Contours in the editor. Ticking contours solves the two-dimensional DC field for the current document on demand and draws the equipotentials over the section, so the effect of moving an electrode is visible before the deck is ever flown. The editor states what it drew — for the ion funnel below, 494 contour lines across 15 levels, field build 0.318 s — and states equally plainly that these are DC potentials only, with RF drives excluded.



The editor's own contour overlay: ticking 'contours' solves the 2-D DC field for the deck on demand and draws the equipotentials over the section, so the effect of moving an electrode is visible in the editor before the deck is ever flown.

11.2 The 3-D Flight Viewer (/flight)

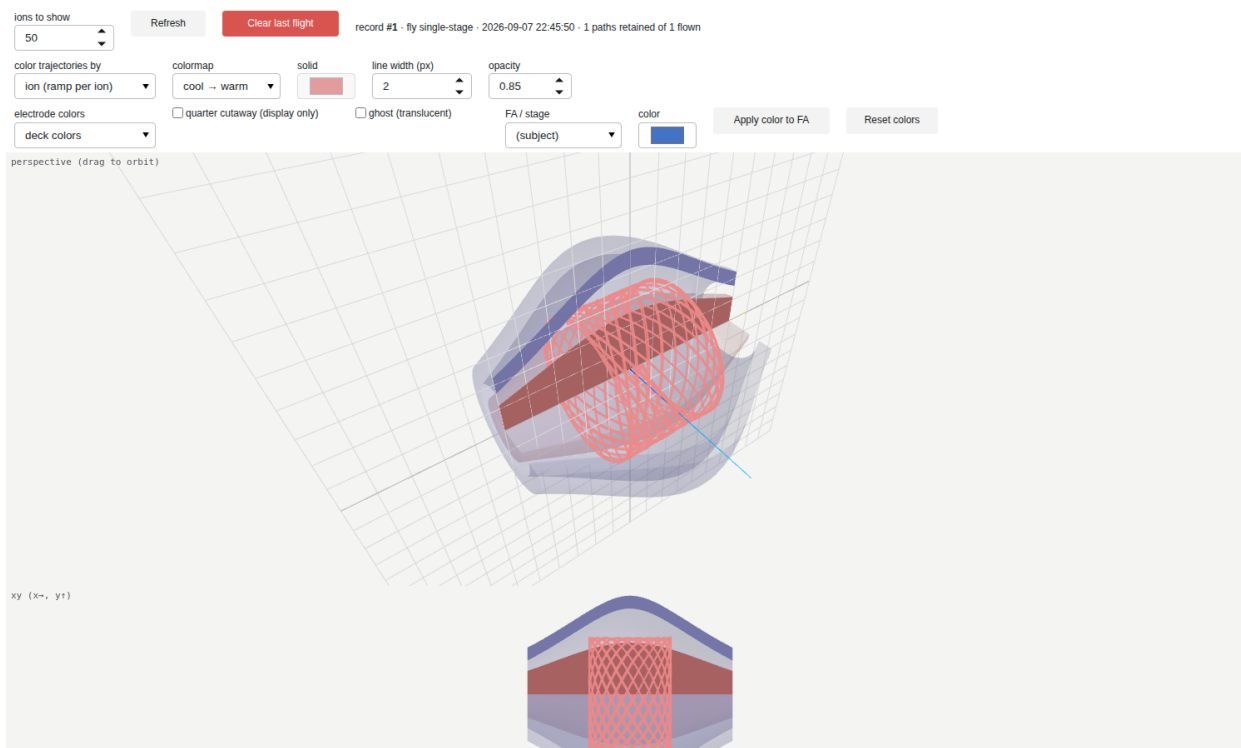
What it is. The 3D Flight Viewer button opens /flight, a read-only view of the LAST flight banked by the dashboard. Fly something, switch to the viewer, press Refresh, and the flown paths appear inside the instrument's electrodes in a scene you can orbit and zoom. The header states which record is being shown and how many paths were retained of those flown, so a thinned view can never be mistaken for the whole flight.



The 3-D Flight Viewer (/flight) on Orbitrap (r-z): the orbits flown in the dashboard, drawn in a rotatable scene with the trap's electrodes. The viewer reads the LAST banked flight -- fly in the dashboard, then press Refresh here. Trajectories are coloured per ion by default; drag to orbit, scroll to zoom, and the perspective is remembered per document across a refresh.

Why a second angle. A single projection of a three-dimensional scene hides an axis. The Orbitrap paths below are the same flight seen after dragging the camera through roughly a quarter turn: the first view shows the orbital motion around the spindle, the second shows the axial oscillation along it, and the instrument is only legible as both at once. The perspective is remembered per instrument, so a refresh returns to the view you left.

3D Flight Viewer



The same Orbitrap (*r-z*) flight after dragging the camera through roughly a quarter turn. A single projection of a 3-D scene hides an axis: here the second angle is what shows the orbits winding along the wire rather than simply circling it.

12. References

- [1] D. A. Dahl, International Journal of Mass Spectrometry 200 (1–3), 3–25 (2000). doi:10.1016/S1387-3806(00)00305-5
- [2] A. Brandt, “Multi-level adaptive solutions to boundary-value problems,” Mathematics of Computation 31 (138), 333–390 (1977). doi:10.2307/2006422
- [3] D. M. Young, “Iterative methods for solving partial difference equations of elliptic type,” Transactions of the American Mathematical Society 76 (1), 92–111 (1954). doi:10.1090/S0002-9947-1954-0059635-7
- [4] A. D. Appelhans and D. A. Dahl, International Journal of Mass Spectrometry 244 (1), 1–14 (2005).
- [5] H. G. Dehmelt, “Radiofrequency spectroscopy of stored ions I: Storage,” Advances in Atomic and Molecular Physics 3, 53–72 (1967).
- [6] D. Gerlich, “Inhomogeneous RF fields: a versatile tool for the study of processes with slow ions,” Advances in Chemical Physics 82, ch. 1 (1992).
- [7] S. A. Shaffer, K. Tang, G. A. Anderson, D. C. Prior, H. R. Udseth and R. D. Smith, “A novel ion funnel for focusing ions at elevated pressure using electrospray ionization mass spectrometry,” Rapid Communications in Mass Spectrometry 11 (16), 1813–1817 (1997).
- [8] W. C. Wiley and I. H. McLaren, “Time-of-flight mass spectrometer with improved resolution,” Review of Scientific Instruments 26 (12), 1150–1157 (1955). doi:10.1063/1.1715212

References [1], [4], [5] and [8] are the documented origins of methods implemented or ported in this toolkit and are recorded as such in the source. All entries above were verified against the publisher record rather than reproduced from memory.

Appendix A. The instrument specification, shape by shape

Section 3 lists the top-level blocks of a specification; this appendix goes one level down, to the part you actually author when you build an instrument: the electrodes and their shapes. Every fragment here is taken from a deck that ships in examples/ and loads through the same `SimSpec.from_json` a user's own file takes.

A.1 The skeleton

An electrode is a named conductor with a list of shapes, a DC value, and optional drive-group memberships. The geometry block holds the electrodes plus the grid declaration (extents, pitch, symmetry, and — for signed frames — `origin_mm`):

The geometry block below is complete and valid as written — it is the analytical quadrupole assembled in full in A.5, which builds, solves, and flies exactly as printed:

```
"geometry": {
  "width_mm": 21.8,
  "height_mm": 21.8,
  "depth_mm": 0.0,
  "mm_per_gu": 0.1,
  "symmetry": {
    "coords": "xyz",
    "planes": {
      "x": "none",
      "y": "none",
      "z": "none"
    }
  },
  "electrodes": [
    {
      "name": "rod_east",
      "dc": 0.0,
      "rf_groups": [
        "RFA"
      ],
      "shapes": [
        {
          "type": "ellipse",
          "cx_mm": 17.344,
          "cy_mm": 10.9,
          "rx_mm": 3.444,
          "ry_mm": 3.444
        }
      ]
    },
    {
      "name": "rod_west",
      "dc": 0.0,
      "rf_groups": [
        "RFA"
      ],
      "shapes": [
        {
          "type": "ellipse",
          "cx_mm": 4.456,
          "cy_mm": 10.9,
          "rx_mm": 3.444,
          "ry_mm": 3.444
        }
      ]
    }
  ]
}
```

```

    },
    {
      "name": "rod_north",
      "dc": 0.0,
      "rf_groups": [
        "RFB"
      ],
      "shapes": [
        {
          "type": "ellipse",
          "cx_mm": 10.9,
          "cy_mm": 17.344,
          "rx_mm": 3.444,
          "ry_mm": 3.444
        }
      ]
    },
    {
      "name": "rod_south",
      "dc": 0.0,
      "rf_groups": [
        "RFB"
      ],
      "shapes": [
        {
          "type": "ellipse",
          "cx_mm": 10.9,
          "cy_mm": 4.456,
          "rx_mm": 3.444,
          "ry_mm": 3.444
        }
      ]
    }
  ],
  "rf_groups": [
    {
      "name": "RFA",
      "frequency_hz": 1000000.0,
      "amplitude_v": 30.0,
      "phase_deg": 0.0,
      "waveform": "sin"
    },
    {
      "name": "RFB",
      "frequency_hz": 1000000.0,
      "amplitude_v": 30.0,
      "phase_deg": 180.0,
      "waveform": "sin"
    }
  ]
}

```

coords is "xyz" for planar/3-D Cartesian builds and "rz" for axisymmetric ones (there x is the axis and y the radius). mm_per_gu is the grid pitch — the resolution of the physics (section 5) — and it is declared, never inferred.

A.2 The shape primitives

Five shape types exist: rect, ellipse, polygon, cutout, and group. All coordinates are millimeters in the deck frame. Any 2-D shape may carry an extrude descriptor giving it a range along one axis in a 3-D build; without one, a 3-D build treats it as a full-depth slab, and a 2-D build ignores extrusion entirely.

rect — an axis-aligned rectangle from its lower-left corner:

```
{"type": "rect", "x_mm": 21.0, "y_mm": 30.0,  
  "width_mm": 14.0, "height_mm": 1.5}
```

This is the detector plate of the reflectron oa-TOF example: a 14 x 1.5 mm slab with its lower-left corner at (21, 30).

ellipse — center plus semi-axes (a circle when $rx = ry$), here with an extrusion making it a 50 mm rod:

```
{"type": "ellipse", "cx_mm": 3.7152, "cy_mm": 2.145,  
  "rx_mm": 1.498, "ry_mm": 1.498,  
  "extrude": {"axis": "z", "lo_mm": 0.0, "hi_mm": 50.0}}
```

This is rod_1 of the native 3-D hexapole example: a round rod of radius 1.498 mm extruded the full 50 mm along z. The cross-section plane is the pair of axes perpendicular to the extrude axis in cyclic order (z -> (x, y)), so cx/cy read as (x, y) here.

polygon — an arbitrary closed solid from its vertex list. The list is implicitly closed: the last vertex connects back to the first, so the first point is not repeated (repeating it is harmless but unnecessary):

```
{"type": "polygon",  
  "points_mm": [[0.0, 0.0], [4.0, 0.0], [4.0, 1.0], [0.0, 2.5]]}
```

The rasterizer classifies polygons reflection-exactly — a polygon and its mirror image land on identical grid cells — which is what lets mirrored geometry be authored once and unfolded (A.4).

cutout — subtract a volume; the child shape is what gets removed:

```
{"type": "rect", "x_mm": 1.0, "y_mm": 168.0,  
  "width_mm": 38.0, "height_mm": 0.9},  
{"type": "cutout", "children": [  
  {"type": "ellipse", "cx_mm": 20.0, "cy_mm": 168.45,  
    "rx_mm": 5.0, "ry_mm": 0.45}]]}
```

Shapes compose in declared order: solids union in, cutouts subtract out. A plate followed by an ellipse cutout is a plate with a beam aperture. A cutout that removes nothing on the grid is refused by name — a no-op subtraction is a declaration failure, not a smaller hole. group simply nests shapes so a repeated sub-assembly can be authored once.

A.3 Signed frames: origin_mm

By default the deck frame starts at (0, 0): node (0,0) of the grid sits at world (0, 0) and every coordinate is non-negative. origin_mm moves that anchor: it declares the world coordinate of grid node (0,0), so a device can be authored centered on the origin with signed coordinates — which is how you naturally measure a multipole. The hexapole example does exactly this:

```
"width_mm": 15.6, "height_mm": 15.6,  
"origin_mm": [-7.8, -7.8],  
...  
{"type": "ellipse", "cx_mm": 3.7152, "cy_mm": 2.145, ...}  
{"type": "ellipse", "cx_mm": -3.7152, "cy_mm": -2.145, ...}
```

The domain reads [-7.8, +7.8] mm in x and y, the six rods sit at +R about (0, 0), the source is born at (0, 0), and the recorded trajectories come back in the same signed frame. origin_mm must sit on the pitch lattice (origin/mm_per_gu integer), and it is part of the geometry identity: two decks differing only in origin are different instruments to the solver cache.

A.4 A symmetry example

Symmetry is declared, never assumed (section 4). Declaring a mirror plane makes the solver store and relax only the non-negative half of that axis — half the memory, roughly half the time — while flight, display and recorded output run in the full, unfolded domain with the mirror plane at coordinate 0. The SLIM 3-D transport example declares one mirror:

```

"geometry": {
  "width_mm": 37.05, "height_mm": 12.35, "depth_mm": 1.6,
  "mm_per_gu": 0.05,
  "symmetry": {"coords": "xyz", "planes": {"z": "mirror"}},
  "electrodes": [
    {"name": "RF1", "shapes": [
      {"type": "rect", "x_mm": 3.277, "y_mm": 0.0,
        "width_mm": 0.406, "height_mm": 0.2}, ... ]},
    ...
  ]
}

```

On the mirrored axis, coordinates mean distance from the mirror plane: the stored half is z in $[0, 0.8]$ and the flight domain reads z in $[-0.8, +0.8]$ with the electrode pattern imaged across $z = 0$ — the two facing SLIM boards authored once. Shapes on a mirrored axis must not reach below 0 (the image supplies that side), and after every mirrored build the toolkit proves, rather than assumes, that the unfolded potentials are bit-symmetric about the declared plane. Because a mirror plane sits at 0 by construction, origin_mm on a mirrored axis is refused — the two mechanisms answer different questions and compose only on distinct axes.

A.5 A fully defined shape set: the analytical quadrupole

Everything above assembled into one complete instrument: four round rods, $r_0 = 3.0$ mm, rod radius $1.148 r_0$ (the dodecapole-minimizing ratio), opposite pairs driven in antiphase at 1 MHz and 30 V. The domain is snapped to the lattice (21.8 mm = 218 cells at 0.1 mm/gu; the remainder of the physical envelope is absorbed at the outer walls, never at a symmetry plane). This deck is reproduced verbatim from a validated file: it passes validate(), builds, and a cold ion launched 0.35 mm off axis is held for the full 160 microsecond flight, oscillating about the center with the expected secular motion.

```

{
  "schema_version": 1,
  "name": "Analytical quadrupole (planar cross-section)",
  "notes": "Four round rods,  $r_0 = 3.0$  mm,  $r_{rod} = 1.148 r_0$  (dodecapole-minimizing ratio), opposite pairs driven in antiphase. Domain snapped to the lattice: 21.8 mm = 218 cells at 0.1 mm/gu.",
  "geometry": {
    "width_mm": 21.8,
    "height_mm": 21.8,
    "depth_mm": 0.0,
    "mm_per_gu": 0.1,
    "symmetry": {
      "coords": "xyz",
      "planes": {
        "x": "none",
        "y": "none",
        "z": "none"
      }
    }
  },
  "electrodes": [
    {
      "name": "rod_east",
      "dc": 0.0,
      "rf_groups": [
        "RFA"
      ],
      "shapes": [
        {
          "type": "ellipse",
          "cx_mm": 17.344,
          "cy_mm": 10.9,
          "rx_mm": 3.444,
          "ry_mm": 3.444
        }
      ]
    }
  ]
}

```

```

    ],
    {
      "name": "rod_west",
      "dc": 0.0,
      "rf_groups": [
        "RFA"
      ],
      "shapes": [
        {
          "type": "ellipse",
          "cx_mm": 4.456,
          "cy_mm": 10.9,
          "rx_mm": 3.444,
          "ry_mm": 3.444
        }
      ]
    },
    {
      "name": "rod_north",
      "dc": 0.0,
      "rf_groups": [
        "RFB"
      ],
      "shapes": [
        {
          "type": "ellipse",
          "cx_mm": 10.9,
          "cy_mm": 17.344,
          "rx_mm": 3.444,
          "ry_mm": 3.444
        }
      ]
    },
    {
      "name": "rod_south",
      "dc": 0.0,
      "rf_groups": [
        "RFB"
      ],
      "shapes": [
        {
          "type": "ellipse",
          "cx_mm": 10.9,
          "cy_mm": 4.456,
          "rx_mm": 3.444,
          "ry_mm": 3.444
        }
      ]
    }
  ],
  "rf_groups": [
    {
      "name": "RFA",
      "frequency_hz": 1000000.0,
      "amplitude_v": 30.0,
      "phase_deg": 0.0,
      "waveform": "sin"
    },
    {
      "name": "RFB",

```

```

        "frequency_hz": 1000000.0,
        "amplitude_v": 30.0,
        "phase_deg": 180.0,
        "waveform": "sin"
    }
]
},
"source": {
    "n_ions": 1,
    "distribution": "point",
    "x0_mm": 11.25,
    "y0_mm": 10.9,
    "mz_list": [
        100.0
    ],
    "charge": 1,
    "ke_lo": 0.0,
    "ke_hi": 0.0,
    "temperature_k": 0.0,
    "seed": 0,
    "tob_span_us": 0.0
},
"integration": {
    "dt_ns": 2.0,
    "t_max_us": 160.0,
    "rec_every": 25
},
"bounds": {},
"collisions": {
    "enabled": false
}
}

```

Paste it into a .json file and load it from the app or from `SimSpec.from_json` to fly it yourself.